

Nexcode: control de gastos personales en Android con arquitectura en capas verificable y recordatorio activo de pagos fijos

Aplicación nativa desarrollada en Kotlin, con dominio multiplataforma, persistencia local y asistente financiero

Universidad de San Buenaventura · Ingeniería de Sistemas
Optativa II — Desarrollo Móvil · Proyecto Integrador · Sexto semestre, 2026

Jeferson Wilderman González Tenjo
Ingeniería de Sistemas
Universidad de San Buenaventura
jwgonzalez@academia.usbbog.edu.co

José Leandro Ocampo Camacho
Ingeniería de Sistemas
Universidad de San Buenaventura
[correo institucional]

III. RESUMEN

Resumen — Este trabajo presenta el diseño, la construcción y la verificación de Nexcode, una aplicación Android nativa para el control de gastos personales dirigida a estudiantes universitarios colombianos. El problema abordado no es la ausencia de soluciones —el mercado es maduro— sino que las alternativas disponibles imponen compromisos que no compensan en el contexto local: las que automatizan el registro exigen cuenta de usuario y almacenamiento en servidores de terceros a cambio de una agregación bancaria que en Colombia funciona de forma parcial, y las gratuitas que operan sin cuenta carecen de recordatorio de pagos fijos. La solución propuesta opera íntegramente en el dispositivo, incorpora un bolsillo de pagos recurrentes con notificaciones del sistema operativo, y añade un asistente financiero, consultable por voz o por escrito, que analiza el gasto del mes. Metodológicamente, el proyecto adopta Clean Architecture materializada en tres módulos Gradle independientes, de modo que la separación entre capas no depende de la disciplina del equipo sino que la impone el compilador. La capa de dominio se implementó como módulo Kotlin Multiplatform y se compila para JVM y JavaScript, lo que permitió detectar dos acoplamientos a la biblioteca de Java que la revisión manual no había advertido. El resultado son 7 631 líneas de Kotlin distribuidas en 80 archivos, verificadas mediante 24 pruebas unitarias que se ejecutan en dos plataformas y 17 casos de prueba de sistema ejecutados sobre dos dispositivos con versiones distintas de Android.

Abstract — This work presents the design, construction and verification of Nexcode, a native Android application for personal expense tracking aimed at Colombian university students. The problem addressed is not the absence of solutions —the market is mature— but that available alternatives impose trade-offs that do not pay off in the local context: those that automate data entry require a user account and third-party storage in exchange for bank aggregation that works only partially in Colombia, while free offline alternatives lack recurring-payment reminders. The proposed solution runs entirely on the device, includes a recurring-payment pocket with operating-system notifications, and adds a financial assistant, available by voice or text, that analyses monthly spending. Methodologically, the project adopts Clean Architecture materialised as three independent Gradle modules, so that layer separation is enforced by the compiler rather than by team

discipline. The domain layer was implemented as a Kotlin Multiplatform module targeting both the JVM and JavaScript, which revealed two couplings to the Java standard library that manual review had missed. The result comprises 7 631 lines of Kotlin across 80 files, verified through 24 unit tests running on two platforms and 17 system test cases executed on two devices running different Android versions.

IV. PALABRAS CLAVE

arquitectura en capas Clean Architecture Kotlin Kotlin Multiplatform Android Jetpack Compose MVVM Room SQLite inversión de dependencias finanzas personales pagos recurrentes trabajo en segundo plano pruebas unitarias asistente conversacional

Index terms — layered architecture, Clean Architecture, Kotlin, Kotlin Multiplatform, Android, Jetpack Compose, MVVM, Room, SQLite, dependency inversion, personal finance, recurring payments, background work, unit testing, conversational assistant.

V. INTRODUCCIÓN

Llevar la cuenta de los gastos propios es una tarea que casi todo el mundo intenta y casi nadie sostiene. La dificultad no es conceptual —sumar y restar— sino de constancia: cualquier aplicación que exija quince segundos por movimiento se abandona en pocos días, y una que exija menos pero no aporte información útil se abandona en pocas semanas. El diseño de una herramienta de este tipo es, por tanto, un problema de ingeniería tanto como de comportamiento.

El estudiante universitario colombiano añade dos condiciones al problema. La primera es que sus ingresos suelen ser irregulares y de baja cuantía —una quincena, una mesada, un trabajo ocasional—, de modo que la diferencia entre llegar o no a fin de mes se juega en cantidades pequeñas que solo se ven al agregarlas. La segunda, y más determinante, es que **una parte considerable de su gasto es fija y previsible**: el arriendo, los servicios, el plan del celular, las suscripciones. Ese gasto no requiere análisis; requiere

memoria. Olvidar el pago del arriendo no es un problema de presupuesto sino de recordatorio.

Este proyecto desarrolla *Nexcode*, una aplicación Android nativa escrita en Kotlin que atiende ese conjunto de necesidades. Se desarrolla en el marco de la asignatura Optativa II — Desarrollo Móvil, lo que impone una condición adicional que conviene declarar desde el principio: el producto no solo debe funcionar, debe **demostrar** el dominio de un conjunto de fundamentos —programación orientada a objetos, estructuras de datos, manejo de excepciones, seguridad ante nulos [22] y arquitectura en capas— de forma que un tercero pueda verificarlo leyendo el código.

Esa doble exigencia condicionó las decisiones técnicas. Una aplicación puede resolver el problema del usuario con una arquitectura monolítica, pero entonces la separación entre lógica de negocio e infraestructura queda como una afirmación del informe que nadie puede comprobar. Por eso la

arquitectura se materializó en módulos independientes cuya separación verifica el compilador, y por eso se documenta cada verificación con el comando que la reproduce.

A. Objetivo general

Desarrollar una aplicación móvil nativa para Android, escrita en Kotlin, que permita a un estudiante universitario registrar y analizar sus gastos personales sin necesidad de cuenta ni conexión a internet, que le recuerde activamente sus pagos fijos, y cuya arquitectura en capas pueda verificarse de forma objetiva.

B. Objetivos específicos

Los objetivos específicos se formularon de modo que cada uno produzca un resultado comprobable, no una afirmación. La Tabla I los relaciona con su evidencia y con la sección del informe que la documenta.

Tabla I
OBJETIVOS ESPECÍFICOS Y SU VERIFICACIÓN

N.º	Objetivo	Resultado verificable	Sección
O-1	Diseñar una arquitectura en tres capas independientes	Una violación deliberada de la regla de dependencias es rechazada por el compilador	11
O-2	Implementar el registro, la edición y la consulta de movimientos con persistencia local	Casos de sistema S-02 a S-07 superados sobre dispositivo	19
O-3	Construir un bolsillo de pagos recurrentes con aviso del sistema operativo	Notificación publicada con la aplicación cerrada y navegación desde ella	19
O-4	Incorporar un asistente que analice el gasto del mes	Respuesta por voz y por escrito generada a partir de datos reales del usuario	18
O-5	Evidenciar los pilares de programación exigidos por el plan de estudios	Fragmentos de código citados literalmente con su archivo de origen	12–16
O-6	Verificar el resultado con pruebas automatizadas	24 pruebas unitarias ejecutadas en dos plataformas y 17 casos de sistema	19

C. Alcance y limitaciones

El proyecto abarca una aplicación funcional para Android con persistencia local, y una capa de dominio portable verificada en una segunda plataforma. **Quedan fuera del alcance:** la sincronización entre dispositivos, la conexión automática con entidades financieras, la exportación de datos, el presupuesto por categoría y la publicación en tienda. Estas ausencias no son omisiones sino recortes conscientes, y se justifican en las secciones correspondientes.

D. Estructura del documento

El informe se organiza en veintidós secciones. Las secciones 5 a 9 plantean el problema y su contexto; las

secciones 10 y 11 documentan el diseño y la arquitectura; las secciones 12 a 16 evidencian los fundamentos de programación aplicados; las secciones 17 y 18 describen la portabilidad y la asistencia mediante inteligencia artificial; y las secciones 19 a 22 recogen la verificación, los resultados y las conclusiones.

VI. PREGUNTA PROBLEMA

La pregunta tiene tres componentes, y esa estructura no es casual: cada uno responde a una limitación concreta identificada en el estado del arte y se traduce en una parte distinta de la solución. La Fig. 1 muestra esa correspondencia.



Cada componente de la pregunta nace de una limitación concreta y se responde con una parte concreta del sistema.

Fig. 1. Descomposición de la pregunta problema. La columna izquierda recoge las limitaciones identificadas en la Sección 8; la central, los componentes de la pregunta; la derecha, la parte del sistema que los atiende. Ningún componente queda sin respuesta y ninguna parte del sistema existe sin justificación.

NOTA SOBRE LA FORMULACIÓN

El tercer componente —que la arquitectura sea verificable y no solo afirmada— es inusual en un enunciado de problema de una aplicación, y responde a la naturaleza académica del proyecto. En un desarrollo comercial bastaría con que el sistema funcionara; aquí debe además poder demostrarse cómo está construido, lo que convierte la verificabilidad en un requisito y no en una virtud opcional.

VII. DESCRIPCIÓN DE LA NECESIDAD

La necesidad atendida se describe a partir de un usuario concreto y no de un perfil abstracto: un estudiante de pregrado que vive con ingresos irregulares, comparte arriendo, se desplaza a diario a la universidad y sostiene un puñado de pagos fijos mensuales. Sobre ese caso se identificaron cuatro necesidades funcionales y dos restricciones de contexto.

A. Necesidades identificadas

Registrar sin fricción. El momento en que se produce un gasto es el único en que el usuario lo recuerda con exactitud, y suele ser un momento incómodo: de pie, con una mano ocupada, saliendo de un local. Si el registro no cabe en ese instante, no ocurre. De aquí se deriva el requisito de que el alta exija un solo dato obligatorio, el monto, y se alcance con un toque desde la pantalla principal.

Entender en qué se va el dinero. Una lista de movimientos responde «qué compré» pero no «en qué se me

fue el mes». La segunda pregunta requiere agregación por categoría, y es la que produce un cambio de comportamiento: nadie reduce un gasto que no sabe que está haciendo.

No olvidar los pagos fijos. Es la necesidad de mayor impacto económico y la peor atendida por las alternativas gratuitas. Un olvido no cuesta el valor del gasto sino el recargo por mora, y a diferencia de los gastos variables, los fijos son enteramente previsibles: la información para avisar ya existe en el sistema desde el momento en que el pago se registra.

Recibir una lectura de la situación. Entre tener los datos y entenderlos hay una distancia que muchos usuarios no recorren. Una interpretación en lenguaje natural —«estás gastando más de lo que ingresas», «esta categoría concentra el 40 % de tus gastos»— convierte una tabla en una conclusión.

B. Restricciones de contexto

Conectividad y datos no garantizados. El usuario objetivo no siempre tiene datos móviles disponibles, y el registro debe funcionar igual sin conexión. Esto descarta cualquier arquitectura que dependa de un servidor para la operación básica.

Sensibilidad de la información. Los movimientos de gasto revelan hábitos, ubicaciones y relaciones personales. Almacenarlos en un servidor de terceros es un costo real que en este caso no se compensa con ninguna ventaja, dado que la agregación bancaria —la única razón de peso para hacerlo— no opera de forma fiable en el país.

Tabla II
DE LA NECESIDAD AL REQUISITO Y A SU IMPLEMENTACIÓN

N.º	Necesidad	Requisito derivado	Implementación
N-1	Registrar sin fricción	Alta con un solo campo obligatorio, a un toque	Botón flotante y título deducido de la categoría
N-2	Entender el gasto	Agregación por categoría y saldo del mes	Gráfico de dona y encabezado de saldo
N-3	No olvidar pagos fijos	Aviso con la aplicación cerrada	Bolsillo de recordatorios y notificación diaria

N-4	Recibir una lectura	Interpretación en lenguaje natural	Asistente financiero
R-1	Operar sin conexión	Ninguna función esencial depende de la red	Persistencia local con Room
R-2	Datos sensibles	Sin cuenta de usuario ni envío a terceros	Todo permanece en el dispositivo

C. Necesidad académica

Junto a la necesidad del usuario existe la del contexto en que el proyecto se desarrolla, y no reconocerla sería deshonesto. El plan de estudios exige evidenciar el uso de estructuras de datos y ciclos, programación orientada a objetos, manejo de excepciones [23] y seguridad ante nulos. Esa exigencia influyó en el diseño, aunque conviene precisar en qué sentido.

No se forzó ninguna construcción artificial para «cumplir un requisito». Las jerarquías selladas [24] de la Sección 13 existen porque el tipo de movimiento y los errores de negocio son conjuntos cerrados y conocidos; el manejo de excepciones de la Sección 14 existe porque el usuario escribe texto libre en un campo numérico; la seguridad ante nulos de la Sección 15 existe porque la base de datos anula la categoría de un movimiento cuando esa categoría se elimina. En todos los casos el pilar académico coincidió con la solución técnica correcta, y donde no habría coincidido se habría documentado la discrepancia.

DELIMITACIÓN HONESTA

Nexcode no resuelve el problema del control de gastos mejor que las aplicaciones consolidadas del mercado, y la Sección 8 lo demuestra con una tabla comparativa desfavorable en varias características. Lo que ofrece es una combinación —operar sin cuenta, recordar los pagos fijos y exponer una construcción auditable— que ninguna de las alternativas revisadas reúne para este caso de uso concreto.

VIII. ESTADO DEL ARTE

El control de gastos personales es un dominio maduro: existen decenas de aplicaciones consolidadas, algunas con más de una década en el mercado. Por ello, la revisión no busca demostrar la ausencia de soluciones —sería falso— sino identificar cómo se distribuyen las que existen, qué compromisos impone cada familia y en qué punto de ese espacio se ubica el presente proyecto.

A. Familias de soluciones identificadas

Las aplicaciones revisadas se agrupan en cuatro familias, definidas por el compromiso principal que asumen entre automatización, costo y control de los datos.

Registro manual local. Monefy [7], Money Manager [8] y Bluecoins [9] almacenan la información en el dispositivo y exigen que el usuario introduzca cada movimiento. A cambio funcionan sin conexión, sin cuenta de usuario y sin costo recurrente. La velocidad de registro es su criterio de diseño dominante: Monefy resuelve el ingreso de un gasto en tres toques, y esa cifra explica buena parte de su adopción, porque una aplicación de gastos que exige quince segundos por movimiento se abandona en pocos días.

Agregación bancaria. Mobills [10], Wallet [11] y Spendee [12] intentan eliminar el registro manual conectándose a las entidades financieras del usuario. El modelo funciona bien donde existe una infraestructura de *open banking* consolidada, pero en Colombia la cobertura es parcial e inestable, de modo que en la práctica el usuario termina registrando manualmente una fracción considerable de sus movimientos. Estas aplicaciones requieren además cuenta de usuario y almacenan los datos en servidores del proveedor.

Metodología presupuestal. YNAB [13] y Actual Budget [14] no se presentan como registradores de gastos sino como implementaciones de un método —el presupuesto de base cero o «de sobres»—. Su propuesta de valor es la disciplina que imponen, no la comodidad. YNAB opera bajo suscripción en dólares, lo que constituye una barrera relevante para un estudiante colombiano; Actual Budget es de código abierto y gratuito, pero traslada al usuario la responsabilidad del despliegue.

Código abierto autoalojado o de referencia. Firefly III [15] es la solución más completa del conjunto revisado y también la de mayor exigencia técnica: requiere un servidor propio, base de datos y mantenimiento. Ivy Wallet [16], por su parte, no destaca por cobertura funcional sino por ser un referente de construcción: una aplicación Android escrita en Kotlin con Jetpack Compose [18] cuyo código puede estudiarse, y que sirvió como referencia de dirección visual para este proyecto.

Tabla III
SOLUCIONES REVISADAS Y SU MODELO DE OPERACIÓN

Aplicación	Familia	Datos	Costo	Código
Monefy	Registro manual local	Dispositivo	Gratuito con compras	Cerrado
Money Manager	Registro manual local	Dispositivo	Gratuito con compras	Cerrado
Bluecoins	Registro manual local	Dispositivo	Gratuito con compras	Cerrado
Mobills	Agregación bancaria	Nube del proveedor	Freemium	Cerrado
Wallet (BudgetBakers)	Agregación bancaria	Nube del proveedor	Freemium	Cerrado
Spendee	Agregación bancaria	Nube del proveedor	Freemium	Cerrado
YNAB	Metodología presupuestal	Nube del proveedor	Suscripción	Cerrado
Actual Budget	Metodología presupuestal	Autoalojado	Gratuito	Abierto
Firefly III	Autoalojado	Servidor propio	Gratuito	Abierto
Ivy Wallet	Referente técnico	Dispositivo	Gratuito	Abierto
Nexcode (este proyecto)	Registro manual local	Dispositivo	Gratuito	Abierto

B. Posicionamiento comparado

La Fig. 2 sitúa las soluciones revisadas en dos ejes que resumen el compromiso descrito: la autonomía que conservan

el usuario y sus datos, y la amplitud funcional. El resultado desmiente cualquier pretensión de haber encontrado un nicho vacío.

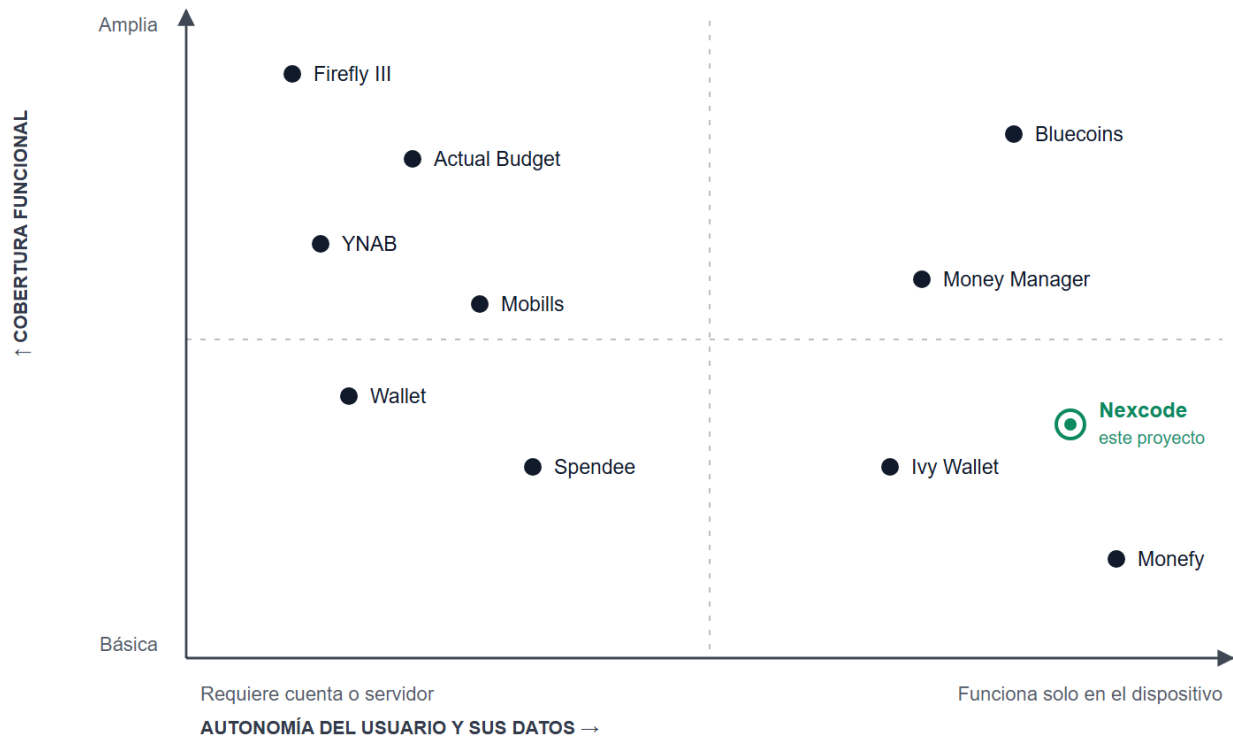


Fig. 2. Mapa de posicionamiento de las soluciones revisadas. El cuadrante superior derecho —máxima autonomía con amplia cobertura— ya está ocupado por Bluecoins. Nexcode se ubica en la zona de alta autonomía y cobertura media, junto a Ivy Wallet y por encima de Monefy, sin superar en funcionalidad a las soluciones consolidadas.

C. Cobertura funcional comparada

La Tabla IV contrasta ocho características relevantes para el caso de estudio. Se incluye a Nexcode en la última fila con

su estado real en el momento de la entrega, incluidas sus ausencias.

Tabla IV
COBERTURA FUNCIONAL COMPARADA

Característica	Monefy	Bluecoins	Mobills	Firefly III	Ivy Wallet	Nexcode
Registro rápido de gastos	Sí	Sí	Sí	Parcial	Sí	Sí
Gráfico por categoría	Sí	Sí	Sí	Sí	Sí	Sí
Recordatorio de pagos fijos	No	Sí	Sí	Sí	No	Sí
Presupuesto por categoría	Parcial	Sí	Sí	Sí	Sí	No
Notificaciones del sistema	No	Sí	Sí	Sí	No	No
Exportar datos	Parcial	Sí	Sí	Sí	Sí	No
Funciona sin cuenta ni servidor	Sí	Sí	No	No	Sí	Sí
Asistente financiero conversacional	No	No	Parcial	No	No	Parcial

La lectura de la tabla es deliberadamente incómoda para el proyecto: Nexcode no supera a Bluecoins ni a Mobills en ninguna característica, y carece de tres que ellas sí ofrecen — presupuesto por categoría, notificaciones del sistema y exportación de datos—. Su asistente financiero funciona hoy mediante un motor de reglas local, con la integración a un agente conversacional remoto preparada pero no conectada.

D. Vacío identificado

Del análisis anterior se desprende que **el vacío no es funcional**. Quien busque hoy una aplicación de control de gastos encuentra alternativas maduras para casi cualquier combinación de necesidades. Sin embargo, la revisión sí permite identificar dos limitaciones transversales que afectan específicamente al usuario objetivo de este proyecto —un estudiante universitario colombiano— y una tercera que atañe al contexto académico en que se desarrolla.

Primera. La automatización que justifica las aplicaciones de agregación bancaria no se materializa en Colombia. El usuario paga el costo del modelo —cuenta obligatoria, datos financieros en servidores de terceros, funciones de pago— sin recibir a cambio la ventaja principal, porque la conexión con las entidades locales es parcial. En ese escenario, una aplicación de registro manual y datos locales no es una solución inferior sino una elección coherente.

Segunda. Las soluciones de código abierto que ofrecen control total sobre los datos exigen infraestructura propia. Firefly III requiere servidor, base de datos y mantenimiento; Actual Budget requiere un despliegue o una sincronización configurada. Para un usuario que solo quiere saber en qué se le fue el mes, ese costo de entrada es prohibitivo.

Tercera. Ninguna de las aplicaciones revisadas fue concebida como objeto de estudio. Las cerradas no exponen su código; las abiertas, aunque legibles, tienen una historia de desarrollo extensa que dificulta usarlas para ilustrar decisiones de diseño puntuales. Para un proyecto integrador cuyo propósito es demostrar el dominio de un lenguaje y de una arquitectura, la construcción propia no es una duplicación de esfuerzo sino el objetivo mismo.

DELIMITACIÓN DEL APORTE

Nexcode no pretende competir en cobertura funcional con las aplicaciones revisadas. Su aporte se ubica en la intersección de tres condiciones que ninguna de ellas satisface simultáneamente para el caso de estudio: **operar sin cuenta ni servidor, incluir el bolsillo de recordatorios de pagos fijos** —ausente en las alternativas gratuitas y locales más simples— y **exponer una arquitectura verificable**, documentada y comprobable en tiempo de compilación, tal como se demuestra en la Sección 11. Formulado con precisión: el resultado del proyecto no es una aplicación mejor que las existentes, sino una aplicación equivalente en su nicho cuya construcción es auditable.

E. Referentes técnicos adoptados

Dos de las soluciones revisadas influyeron directamente en decisiones del proyecto. De **Monefy** se adoptó el criterio de velocidad de registro: el formulario de Nexcode se alcanza con un solo toque desde la pantalla principal y exige un único campo obligatorio, el monto, porque el título se deduce de la categoría cuando el usuario no lo escribe. De **Ivy Wallet** se tomó la referencia de dirección visual y la confirmación de que Kotlin con Jetpack Compose es una combinación viable para este tipo de aplicación.

La identidad visual final, sin embargo, no proviene de ninguna de las aplicaciones revisadas sino del sistema de diseño de *NEXCODE Banca digital*, el otro producto desarrollado por el equipo en la misma asignatura, con el fin de que ambas aplicaciones se reconozcan como parte de una misma familia.

F. Referencias de esta sección

IX. TECNOLOGÍAS UTILIZADAS

La selección tecnológica siguió tres criterios: preferencia por las herramientas oficiales de la plataforma Android, minimización del número de dependencias externas, y compatibilidad verificada por construcción y no por documentación. Este último criterio resultó decisivo, como se explica al final de la sección.

A. Lenguaje y herramientas de construcción

Tabla V
LENGUAJE, CONSTRUCCIÓN Y ENTORNO

Componente	Versión	Función en el proyecto
Kotlin	2.3.21	Lenguaje único de las tres capas
Kotlin Multiplatform	2.3.21	Compilación del dominio para JVM y JavaScript
Android Gradle Plugin	8.7.3	Construcción de los módulos Android
Gradle	8.14.5	Sistema de construcción y catálogo de versiones
JDK	21 LTS	Compilación; objetivo de la JVM
compileSdk / targetSdk	35	API de Android sobre la que se compila
minSdk	26	Android 8.0; permite java.time sin azucarado
Android Studio	Quail (2026.1)	Entorno de desarrollo

B. Bibliotecas por capa

La Tabla VI agrupa las dependencias por la capa que las consume, lo que hace visible una propiedad de la arquitectura: el dominio depende de dos bibliotecas y ninguna es de Android.

Tabla VI
BIBLIOTECAS UTILIZADAS, POR CAPA

Capa	Biblioteca	Versión	Para qué
Dominio	kotlinx-coroutines	1.9.0	Flujos reactivos y funciones suspendidas
kotlinx-datetime	0.8.0	Fechas multiplataforma	

Datos	Room	2.8.4	Persistencia sobre SQLite con migraciones
org.json	Android	Construcción del payload del asistente	
Presentación	Compose BOM	2025.01.00	Coordinación de versiones de Compose
Material 3	via BOM	Componentes y gestos de deslizamiento	
Navigation Compose	2.8.5	Navegación entre pantallas	
Lifecycle	2.8.7	ViewModel y recolección con ciclo de vida	
Activity Compose	1.9.3	Integración de Compose y permisos	
WorkManager	2.10.5	Revisión diaria de pagos y notificaciones	
API del sistema	SpeechRecognizer	Android	Reconocimiento de voz sin dependencia externa
TextToSpeech	Android	Síntesis de voz en español	
Pruebas	kotlin.test · JUnit	4.13.2	Pruebas unitarias en las dos plataformas

C. Herramientas de verificación

Además de las bibliotecas que forman parte del producto, el proyecto se apoyó en un conjunto de herramientas de verificación: el **compilador de Kotlin** como detector de violaciones de arquitectura, **Android Lint** para el análisis estático, y las utilidades del SDK —adb, uiautomator, logcat y dumpsys— para recoger evidencia sobre el dispositivo emulado. La Sección 19 documenta su uso.

```
e: java.lang.IllegalArgumentException: Provided Metadata instance has
version 2.4.0, while maximum supported version is 2.3.0.
To support newer versions, update the kotlin-metadata-jvm library.
```

Se probaron tres versiones de Room —2.6.1, 2.7.2 y 2.8.4— y ninguna resolvió la incompatibilidad, porque el límite lo impone el procesador y no la versión de la biblioteca. La combinación verificada por construcción es **Kotlin 2.3.21 con Room 2.8.4**.

CRITERIO ADOPTADO

Ninguna versión de la Tabla V o XXXII se tomó de la documentación de su fabricante: todas se fijaron probando la construcción completa. Una matriz de compatibilidad publicada describe lo que debería funcionar; un BUILD SUCCESSFUL describe lo que funciona.

D. Una restricción de compatibilidad no documentada

La elección de Kotlin 2.3.21 y no de la versión más reciente disponible merece explicación, porque no fue una preferencia sino una restricción descubierta al construir. El procesador de anotaciones de Room [5] no admite los metadatos que genera Kotlin 2.4, y falla con el siguiente mensaje:

X. DISEÑO DE LA APLICACIÓN

A. Del logotipo a la paleta

El punto de partida no fue una guía de estilo abstracta sino el archivo del logotipo. Se analizaron los píxeles del símbolo —la letra X formada por dos flechas— descartando los tonos neutros del fondo y agrupando el resto por frecuencia. El resultado fueron cuatro colores dominantes que constituyen la paleta de marca de la Tabla VII.

Tabla VII
PALETA DE MARCA MEDIDA SOBRE EL LOGOTIPO

Muestra	Nombre	Valor	Origen en el logotipo	Uso en la aplicación
	Azul profundo	#104080	Texto «NEXCODE»	Inicio de los degradados, sombras
	Azul de marca	#1080D0	Brazo izquierdo de la X	Color primario, acciones, selección
	Cian	#5FD8E8	Cruce de las dos flechas	Estados intermedios, acentos
	Verde menta	#40E0A0	Brazo derecho de la X	Ingresos, cierre de los degradados
	Verde profundo	#17A97B	Derivado del verde menta	Ingresos en texto sobre blanco

El logotipo se incorporó además como recurso gráfico. El archivo original tiene fondo blanco opaco, lo que impedía usarlo sobre cualquier superficie de color, así que se procesó por programa para volver transparente el fondo: se recortó al contenido real y se calculó el canal alfa a partir de la

luminosidad de los píxeles neutros. De ahí salieron dos activos: el símbolo aislado, que sirve como icono de lanzador adaptativo y encabeza la pantalla de Perfil, y el logotipo completo con su texto.

B. El fondo blanco y el problema de contraste

La aplicación se diseñó con fondo blanco. Esa decisión, aparentemente inocua, invalidó uno de los colores de marca para el uso más frecuente: el verde menta #40E0A0 sobre blanco no alcanza contraste suficiente para texto pequeño, y los ingresos son precisamente texto pequeño que el usuario debe poder leer de un vistazo.

La solución no fue abandonar el verde de la marca sino derivar una segunda variante. El verde menta se conserva para **rellenos** —el cierre del degradado de la tarjeta de saldo, los puntos indicadores sobre fondo azul—, donde su claridad es una virtud; y el verde profundo #17A97B se usa para **texto** sobre blanco. Son el mismo color de marca en dos densidades, no dos colores distintos.

CRITERIO
Cada color de la interfaz existe en la densidad que su fondo exige. Un tono que funciona sobre azul no tiene por qué funcionar sobre blanco, y forzarlo produce texto que se lee mal aunque la paleta sea «la correcta».

C. Lenguaje visual heredado de la banca

El equipo desarrolló en la misma asignatura la aplicación *NEXCODE Banca digital*. Para que ambos productos se reconozcan como una familia, Nexcode adoptó su lenguaje de formas leyendo directamente sus recursos gráficos: los degradados, los radios de esquina y las elevaciones no se aproximaron a ojo, se copiaron de los archivos originales. La Tabla VIII establece la correspondencia.

Tabla VIII CORRESPONDENCIA CON LOS RECURSOS DE NEXCODE BANCA DIGITAL		
Recurso del banco	Especificación	Componente en

		Nexcode
fondo_encabezado	Degradado azul, esquinas inferiores de 34 dp	Encabezado de la pantalla principal
fondo_tarjeta_saldo	Tres tonos del logotipo, radio 26 dp, sombra 10 dp	Tarjeta del bolsillo de recordatorios
forma_boton_degradado	Azul profundo → azul → verde, radio 18 dp	Botones principales de toda la app
nex_borde	Borde #E2E9F2 de 1 dp	Tarjetas y filas de movimientos
nex_texto	#0E1C33 · #4A5B72 · #8494A8	Jerarquía tipográfica en tres niveles

Los tres degradados del sistema, todos derivados de la misma paleta, cumplen funciones distintas:

Tabla IX DEGRADADOS DEL SISTEMA Y SU FUNCIÓN		
Muestra	Función	Composición
	Encabezado	Solo azules: el saldo debe leerse en blanco sobre él
	Tarjeta de saldo	Los tres tonos del logotipo, de extremo a extremo
	Botones	Cierra en verde profundo para que el texto blanco resalte

D. Estructura de navegación

La aplicación tiene seis pantallas organizadas en dos niveles. Cuatro son destinos permanentes, accesibles desde la barra inferior con iconos vectoriales propios; las otras dos son pantallas completas que ocultan la barra porque exigen atención exclusiva. La Fig. 3 muestra el mapa.

DESTINOS DE LA BARRA INFERIOR

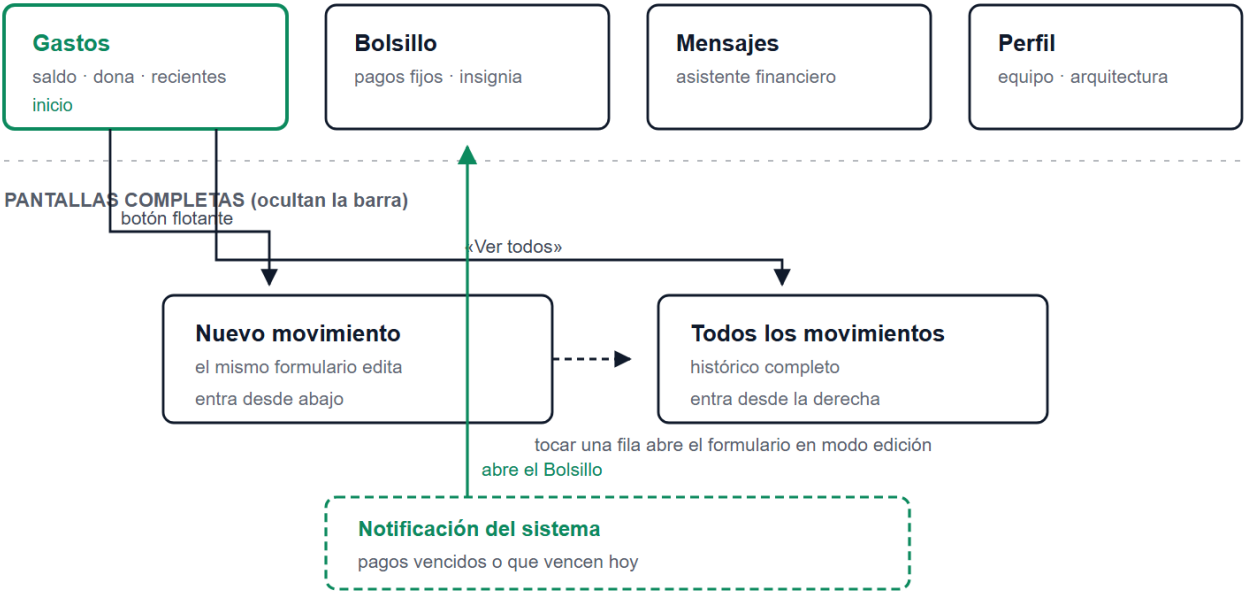


Fig. 3. Mapa de navegación. Las cuatro pestañas conservan su estado al alternar entre ellas. El formulario es único: sin argumento crea un movimiento, con identificador lo edita. La notificación del sistema constituye una cuarta vía de entrada que lleva directamente al Bolsillo.

E. Anatomía de la pantalla principal

La pantalla de Gastos concentra las decisiones de composición del proyecto y sigue la estructura del inicio del

banco: un bloque de marca de ancho completo en la parte superior y contenido en tarjetas debajo. La Fig. 4 identifica sus regiones y el porqué de cada una.

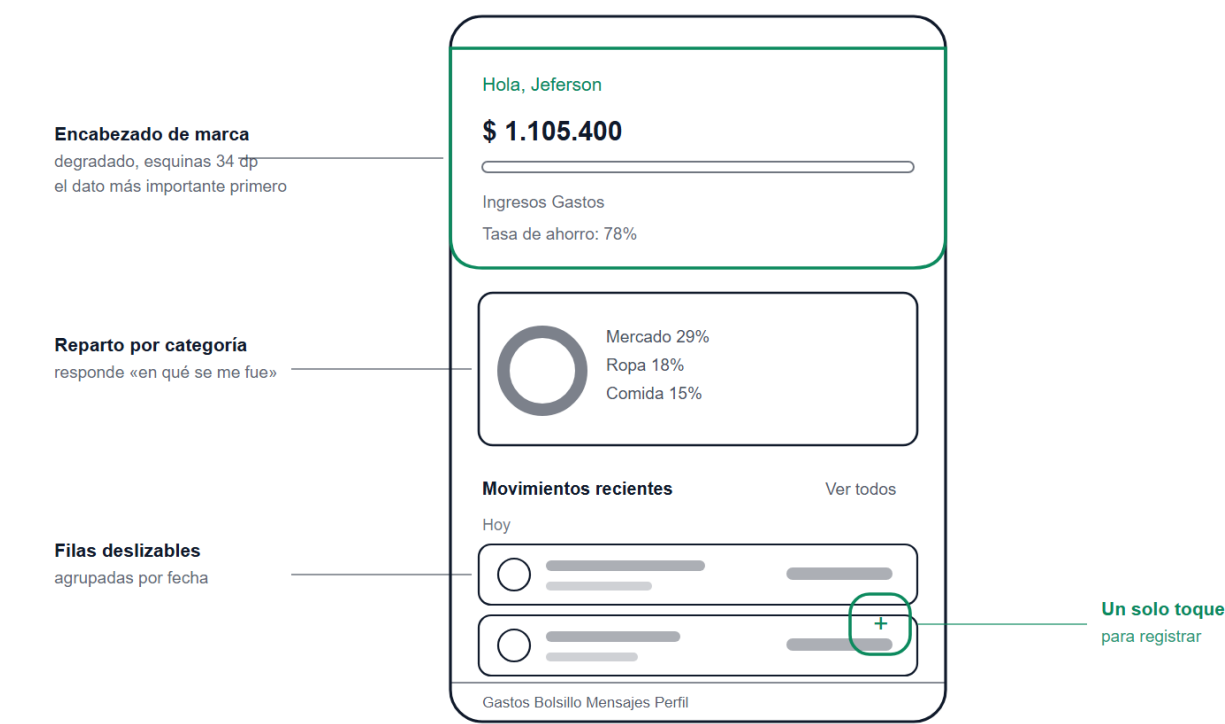


Fig. 4. Anatomía de la pantalla principal. El orden vertical responde a tres preguntas en secuencia: cuánto tengo, en qué se me fue, y qué hice últimamente. El botón de registro permanece accesible sobre el contenido porque es la acción que más veces se repite.

F. Componentes reutilizables

La interfaz se construyó sobre un conjunto reducido de componentes propios, de modo que un cambio de estilo se aplica en un solo lugar. La Tabla X los enumera.

AnimatedMoneyText	Importe con contador animado	Cuatro pantallas
GradientButton	Botón de marca con respuesta al pulsar	Todas las acciones principales
NexcodeCard	Tarjeta blanca con borde suave	Mensajes · Perfil

Tabla X
COMPONENTES PROPIOS DE LA INTERFAZ

Componente	Responsabilidad	Dónde se usa
BalanceHeader	Encabezado con degradado y saldo	Gastos
CategoryDonutChart	Gráfico de dona con leyenda	Gastos
TransactionRow	Fila de movimiento con inicial, título y monto	Gastos · Movimientos
DateHeader	Encabezado de día con total neto	Gastos · Movimientos
SwipeToActionBox	Contenedor con gestos de deslizamiento	Gastos · Movimientos · Bolsillo

G. Movimiento

Las animaciones no se añadieron como adorno: cada una comunica algo que de otro modo el usuario tendría que deducir. Un saldo que salta de una cifra a otra no dice si subió o bajó; un saldo que recorre el camino sí. Una fila que desaparece de golpe no explica qué pasó con las de abajo; una lista que se reacomoda sí. La Tabla XI documenta las doce implementadas.

Tabla XI
ANIMACIONES IMPLEMENTADAS

Elemento	Tipo	Duración	Qué comunica
Saldo y montos	Contador interpolado	700 ms	La dirección y magnitud del cambio
Barra de proporción	Interpolación de peso	800 ms	Cómo se desplaza el equilibrio ingresos/gastos
Gráfico de dona	Barrido de arcos	900 ms	Que los sectores son partes de un total
Cambio de pestaña	Fundido y desplazamiento	320 ms	Continuidad entre secciones hermanas
Nuevo movimiento	Entrada desde abajo	320 ms	Que es una hoja sobre el contenido, no otra sección
Histórico	Entrada desde la derecha	320 ms	Profundidad: se entra y se vuelve

Botón flotante	Resorte al aparecer	Resorte	Presencia sin brusquedad
Icono de pestaña activa	Escala 1,15 con rebote	Resorte	Confirmación del destino elegido
Botones	Compresión al 0,97	Resorte	Respuesta táctil inmediata
Pago vencido	Latido del indicador	900 ms ↔	Urgencia sin recurrir a un diálogo
Formulario del bolsillo	Despliegue en altura	300 ms	De dónde sale y a dónde vuelve
Mensajes de error	Desvanecido con expansión	200 ms	Aparición sin salto del resto del formulario

H. Gestos de deslizamiento

Las listas admiten acciones por deslizamiento, con dos criterios de diseño. Primero, **el fondo anticipa la acción**: al arrastrar aparece un color y un icono que crecen conforme

avanza el gesto, de modo que el usuario ve qué va a ocurrir antes de soltar. Segundo, **el umbral es del 35 % del ancho**, lo bastante alto para que desplazar la lista no dispare un borrado accidental.

Tabla XII
ACCIONES POR DESLIZAMIENTO

Pantalla	Hacia la derecha	Hacia la izquierda	Tras soltar
Movimientos	—	Eliminar	La fila se descarta y la lista se reacomoda
Bolsillo	Registrar el pago	Eliminar el recordatorio	Al pagar, la fila vuelve a su sitio con rebote

La diferencia del comportamiento tras soltar es intencionada. Eliminar retira el elemento, de modo que la fila se descarta. Registrar un pago no lo elimina —el recordatorio sigue existiendo con una fecha nueva—, así que la fila regresa a su posición con una animación de resorte: el movimiento comunica que la acción se ejecutó pero el elemento permanece.

I. Accesibilidad

Se atendieron tres aspectos. El **contraste**, mediante la doble densidad del verde descrita en el apartado B, y verificando que el texto secundario mantenga distancia suficiente del fondo blanco. Las **descripciones de contenido**, presentes en todos los iconos de navegación y en el botón de registro, para que un lector de pantalla los anuncie. Y el **color como refuerzo y no como único portador de información**: el estado de un pago fijo se comunica a la vez por color, por etiqueta textual —«Vencido», «Vence hoy», «En 2 días»— y por la posición en la lista, de modo que un usuario con dificultad para distinguir tonos conserva toda la información.

LIMITACIÓN RECONOCIDA

La aplicación solo ofrece tema claro. Se implementó primero un tema oscuro y se sustituyó por el claro a petición del cliente, pero el sistema de color está construido sobre variables semánticas, de modo que reintroducir el tema oscuro consiste en definir un segundo juego de valores y no en reescribir las pantallas. Tampoco se han probado tamaños de fuente ampliados del sistema.

A. Introducción

El objetivo arquitectónico de Nexcode fue que la lógica de negocio de la aplicación —qué es un gasto válido, cómo se calcula un saldo, cuándo vence un pago fijo— pudiera existir, compilarse y probarse sin ninguna referencia al sistema operativo Android. Esta restricción, aparentemente severa, es la que produce las tres propiedades que se buscaban: pruebas unitarias que corren en la máquina virtual de Java en segundos y no en un emulador; libertad para sustituir la base de datos o el framework de interfaz sin reescribir las reglas del negocio; y un límite explícito y verificable entre lo que es negocio y lo que es infraestructura.

La solución adoptada combina dos patrones complementarios. **Clean Architecture** [1] organiza el sistema en capas concéntricas donde las dependencias apuntan siempre hacia el centro, ocupado por las reglas de negocio. **MVVM** [25] organiza internamente la capa más externa, separando la vista de su estado observable [2]. Conviene distinguirlos desde el inicio: Clean Architecture describe la estructura global del proyecto; MVVM describe únicamente cómo se organiza la capa de presentación.

El proyecto comprende 7 631 líneas de Kotlin distribuidas en 80 archivos fuente, y su distribución por capas se muestra en la Tabla XIII. La capa de dominio se implementó como un módulo **Kotlin Multiplatform** [19], de modo que la misma lógica de negocio compila para la máquina virtual de Java y para JavaScript; la Sección 17 documenta esa migración en detalle.

XI. ARQUITECTURA DE LA APLICACIÓN

Tabla XIII
DISTRIBUCIÓN DEL CÓDIGO FUENTE POR CAPA

Módulo Gradle	Capa	Archivos .kt	Naturaleza
:domain	Dominio	29	Módulo Kotlin Multiplatform (JVM + JS)
:data	Datos	18	Biblioteca Android
:app	Presentación	33	Aplicación Android

B. Visión general y regla de dependencias

Cada capa se implementó como un módulo Gradle [26] separado en lugar de como un paquete dentro de un módulo único. La diferencia es determinante: con paquetes, cualquier programador puede escribir `import androidx.room.Entity` dentro de una clase de negocio y nadie se entera hasta la revisión de código; con módulos, ese `import` no resuelve

porque la dependencia no está declarada, y la compilación falla. La regla de dependencias deja de ser una convención documentada y pasa a ser una restricción del sistema de construcción.

La Fig. 5 muestra los tres módulos y la única dirección permitida para las dependencias.

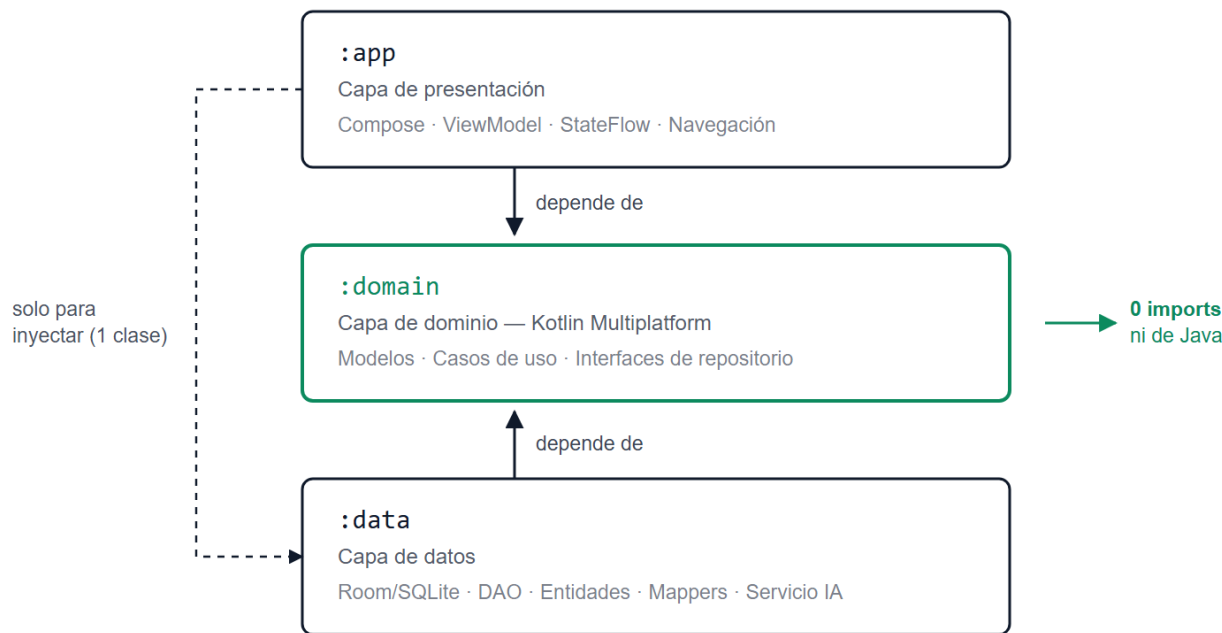


Fig. 5. Los tres módulos y la dirección de sus dependencias. `:domain` no depende de ningún otro módulo, por lo que puede compilarse y probarse de forma aislada. La flecha punteada indica el único acceso de `:app` a `:data`: el contenedor de dependencias, que inyecta las implementaciones concretas.

La declaración efectiva de estas dependencias, tomada de los archivos `build.gradle.kts`, es la siguiente:

```
:domain → (ninguna dependencia de módulo)
:data → implementation(project(":domain"))
:app → implementation(project(":domain"))
      implementation(project(":data"))
```

C. Inversión de dependencias

La Fig. 5 plantea una aparente paradoja: si `:data` depende de `:domain`, ¿cómo puede el dominio guardar información en una base de datos que no conoce? La respuesta es el principio de inversión de dependencias [1]: el dominio no depende de la base de datos, sino que *declara el contrato* que cualquier

mecanismo de persistencia debe cumplir, y la capa de datos lo implementa.

En Nexcode ese contrato es la interfaz `TransactionRepository`, declarada en el dominio. La clase `TransactionRepositoryImpl`, que vive en la capa de datos y sí conoce Room, la implementa. El resultado es que la dirección de la dependencia en tiempo de compilación (hacia arriba, del detalle a la abstracción) es opuesta al flujo de control en tiempo de ejecución (hacia abajo, de la abstracción al detalle). La Fig. 6 muestra ambos sentidos.

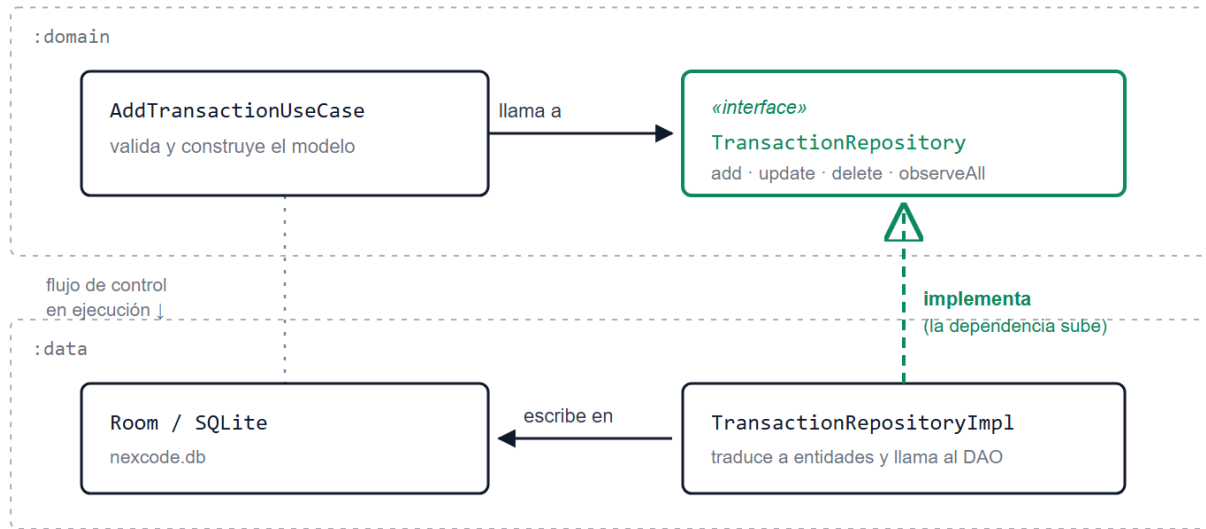


Fig. 6. Inversión de dependencias. En tiempo de ejecución el control desciende del caso de uso hacia SQLite; en tiempo de compilación la dependencia asciende, porque es la implementación la que conoce el contrato y no al revés. Sustituir Room por otro motor solo obliga a reescribir la caja inferior derecha.

La Tabla XIV enumera los cinco contratos declarados en el dominio y las clases que los cumplen. Obsérvese que FinancialAdvisor sigue el mismo patrón aplicado a un servicio remoto: el dominio pide consejo financiero sin saber si lo produce un motor de reglas local o un agente conversacional externo.

Tabla XIV
CONTRATOS DEL DOMINIO Y SUS IMPLEMENTACIONES

Contrato (en :domain)	Implementación (en :data)	Mecanismo
TransactionRepository	TransactionRepositoryImpl	Room · TransactionDao
AccountRepository	AccountRepositoryImpl	Room · AccountDao
CategoryRepository	CategoryRepositoryImpl	Room · CategoryDao
RecurringPaymentRepository	RecurringPaymentRepositoryImpl	Room · RecurringPaymentDao
FinancialAdvisor	NexcodeAIService	Motor de reglas local o webhook HTTP

1) Capa de dominio:

Es el centro del sistema y el único módulo que no depende de nada. Contiene tres tipos de elementos. Los **modelos** representan los conceptos del negocio y protegen sus propias invariantes: Transaction rechaza en su bloque init un monto no positivo o un título vacío, de modo que es imposible que exista en memoria una transacción inválida. El valor monetario se modela con Money, una *value class* [27] que almacena centavos en un Long para evitar los errores de redondeo del punto flotante binario.

Las fechas se manejan con **kotlinx-datetime** [20] y no con java.time, y la hora actual llega por la interfaz inyectable RelojDominio en lugar de leerse de un reloj estático: conocer la hora es una dependencia del entorno, no una regla de negocio, y esa inyección es la que hace deterministas las pruebas.

Los **contratos** son las interfaces de repositorio [4] ya descritas. Los **casos de uso** son clases con una única responsabilidad, invocables como funciones gracias al operador invoke. La Tabla XV los agrupa por paquete.

D. Composición de cada capa

Tabla XV
CASOS DE USO IMPLEMENTADOS, POR PAQUETE

Paquete	Casos de uso	N.º
usecase.transaction	AddTransactionUseCase · UpdateTransactionUseCase · DeleteTransactionUseCase · GetTransactionsUseCase · GetTransactionByIdUseCase	5
usecase.analytics	CalculateBalanceUseCase · GetCategorySummaryUseCase · ObserveMonthlyReportUseCase	3
usecase.recurring	AddRecurringPaymentUseCase · PayRecurringPaymentUseCase · ObserveRecurringPocketUseCase · ToggleRecurringPaymentUseCase · DeleteRecurringPaymentUseCase	5

usecase.catalog	ObserveCategoriesUseCase · ObserveAccountsUseCase	2
usecase.ai	RequestFinancialAdviceUseCase	1

Un caso de uso puede componer otros. `ObserveMonthlyReportUseCase` combina `CalculateBalanceUseCase` y `GetCategorySummaryUseCase` sobre el mismo conjunto de movimientos filtrado por mes, lo que garantiza que el saldo mostrado en el encabezado y el gráfico de reparto por categoría no puedan contradecirse: ambos provienen del mismo cálculo.

Una decisión que conviene explicitar es el **tratamiento de errores**. Los casos de uso no propagan excepciones hacia la interfaz: las atrapan y las traducen a un tipo sellado de resultado. `AddTransactionUseCase` devuelve `AddTransactionResult`, cuyas variantes son `Success`, `InvalidAmount`, `MissingField`, `NotFound` y `Failure`. La capa de presentación no maneja excepciones: maneja estados. Esto elimina toda una clase de fallos en tiempo de ejecución y hace exhaustiva la expresión `when` que los procesa.

2) Capa de datos:

Implementa la persistencia con Room sobre SQLite [28] y encapsula por completo ese hecho. Contiene las **entidades** anotadas con `@Entity`, los **DAO** con las consultas, los **convertidores** de tipos de `java.time`, los **mappers** que traducen entre entidades y modelos de dominio, y las **implementaciones** de los repositorios.

El módulo expone al exterior una sola clase pública, `DataProvider`, que construye las implementaciones y las devuelve tipadas como los contratos del dominio. Esta fábrica no es un adorno: durante el desarrollo, una versión anterior del contenedor de dependencias construía la base de datos directamente desde `:app` y el proyecto dejó de compilar con el error `Cannot access 'androidx.room.RoomDatabase'`, porque Room está declarado como `implementation` y no se propaga transitivamente. El error del compilador señaló correctamente una fuga arquitectónica, y la solución fue mover el ensamblaje al interior de la capa que lo posee.

La base de datos se versiona explícitamente. La incorporación del bolsillo de pagos recurrentes elevó el

esquema de la versión 1 a la 2 mediante una migración escrita a mano (`MIGRATION_1_2` [29]) en lugar de una recreación destructiva, para no perder los movimientos ya registrados por el usuario al actualizar la aplicación.

3) Capa de presentación:

Construida con Jetpack Compose y organizada según MVVM. Cada pantalla tiene un `ViewModel` que expone un único `StateFlow` [6] de solo lectura con una *data class* inmutable de estado —por ejemplo `HomeUiState`—. La vista observa ese estado y emite eventos; nunca lo modifica directamente ni conoce los repositorios.

Esta capa aloja además todo lo que depende del sistema operativo y que por definición no puede compartirse: el trabajo en segundo plano con **WorkManager** [17] que revisa a diario el bolsillo y publica la notificación de los pagos vencidos, el canal de notificaciones y la solicitud del permiso `POST_NOTIFICATIONS` [30]. El trabajador consulta el dominio a través del contenedor y no conoce Room ni la base de datos.

El ensamblaje corre por cuenta de `AppContainer`, un localizador de servicios explícito que crea los casos de uso a partir de las implementaciones que le entrega `DataProvider`, y de `NexcodeViewModelFactory`, que inyecta esos casos de uso en cada `ViewModel`. Se descartó Hilt de forma deliberada: en un proyecto de este tamaño, un contenedor escrito a mano hace visible en un solo archivo toda la cadena de dependencias, lo que resulta más defendible en una sustentación académica que un grafo generado por anotaciones.

E. Recorrido de una operación

Para hacer tangible la separación descrita, la Fig. 7 sigue el registro de un gasto desde que el usuario escribe el monto hasta que la lista de movimientos se actualiza en pantalla. Es relevante que el dato cambie de forma en cada frontera: la interfaz maneja texto crudo, el dominio maneja modelos validados y la capa de datos maneja filas.

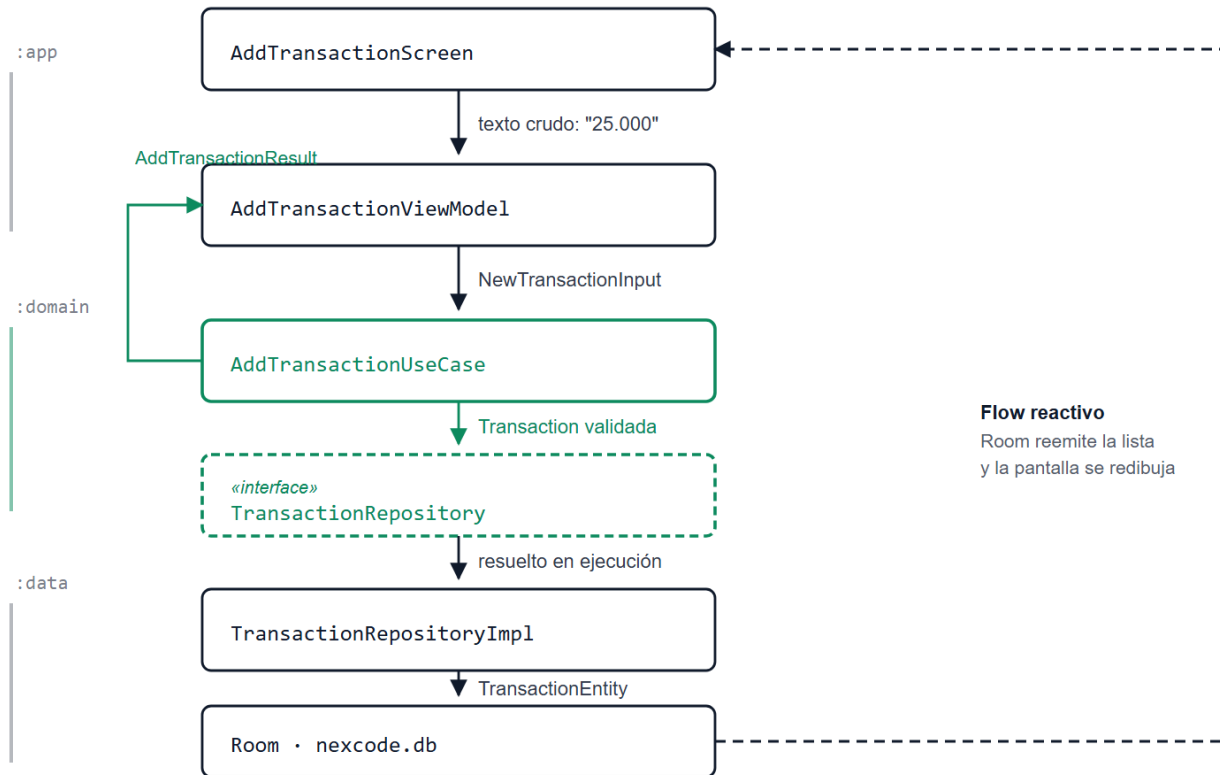


Fig. 7. Registro de un gasto a través de las tres capas. El texto que escribe el usuario solo se convierte en un modelo de dominio después de validarse en el caso de uso; el error, si lo hay, regresa como estado y no como excepción. La ruta punteada de la derecha es la actualización reactiva: escribir en la base de datos hace que Room reemita la lista y la pantalla se redibuje sin intervención explícita.

F. Modelo de datos

El esquema relacional consta de cuatro tablas. Las claves foráneas declaran políticas de borrado distintas según la semántica del negocio: eliminar una cuenta arrastra sus movimientos (CASCADE), mientras que eliminar una categoría

deja los movimientos sin categoría (SET NULL). Esta segunda decisión es la que obliga a que `categoryId` sea de tipo `Long` en el modelo de dominio [3], y por tanto la que justifica el uso sistemático de operadores de seguridad ante nulos en las capas superiores.

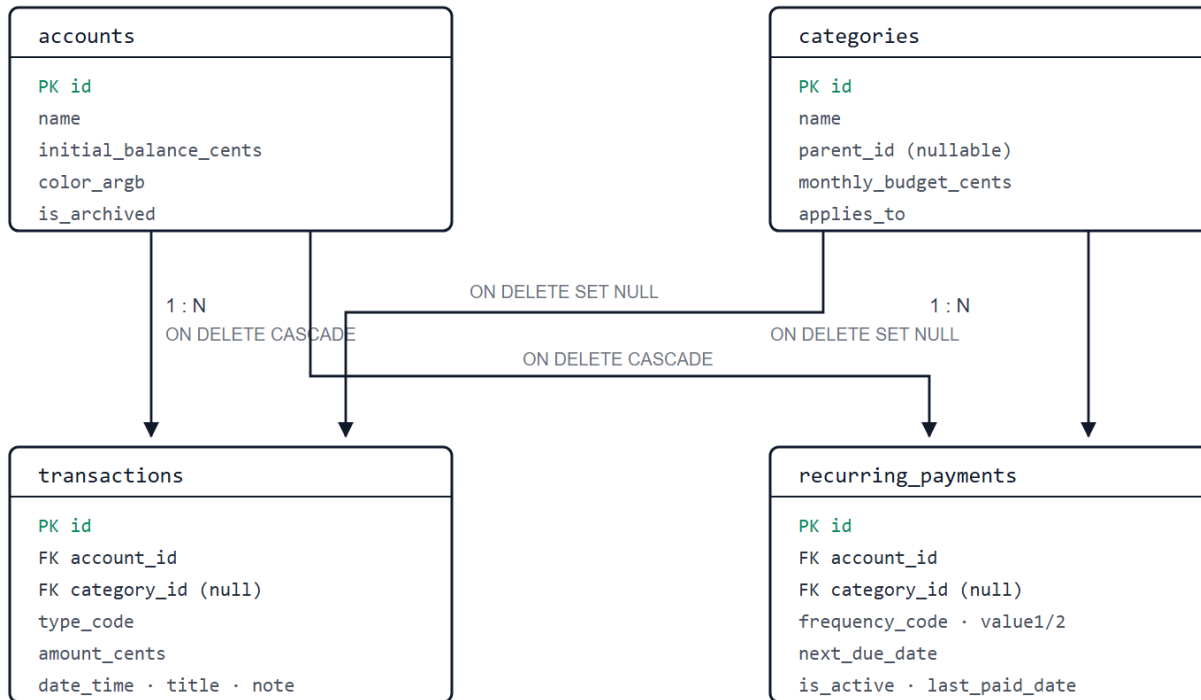


Fig. 8. Modelo entidad-relación de nexcode.db (versión 2 del esquema). Los importes se almacenan como enteros en centavos; las fechas, como texto ISO o milisegundos según el tipo. La política SET NULL sobre category_id es la razón de que ese campo sea opcional en todo el dominio.

G. Verificación de la arquitectura

Una arquitectura documentada no es necesariamente una arquitectura vigente. Para comprobar que la separación descrita

se sostiene en el código entregado, se definieron cinco verificaciones reproducibles sobre el repositorio.

El código de la prueba 4 y la salida obtenida son los siguientes:

```
// domain/src/main/kotlin/com/nexcode/gastos/domain/PruebaDeCapa.kt
package com.nexcode.gastos.domain

import androidx.room.Entity // intento de acoplar el dominio a Room

@Entity class PruebaDeCapa
$ ./gradlew :domain:compileKotlin

e: PruebaDeCapa.kt:3:8 Unresolved reference 'androidx'.
e: PruebaDeCapa.kt:5:2 Unresolved reference 'Entity'.

BUILD FAILED in 1s
```

El archivo fue eliminado tras la comprobación y el proyecto vuelve a construirse sin errores. La Tabla XVI resume las cuatro verificaciones.

Tabla XVI
VERIFICACIONES DE AISLAMIENTO ENTRE CAPAS

N.º	Propiedad verificada	Resultado esperado	Obtenido
1	Dominio libre de Android	0 imports	0
2	Datos encapsulados	≤ 1 import en :app	1 (contenedor)
3	Pruebas en JVM	Ejecución sin emulador	24/24
4	Portabilidad del	Las mismas	24/24

Tabla XVII
DECISIONES ARQUITECTÓNICAS Y ALTERNATIVAS DESCARTADAS

Decisión	Alternativa descartada	Motivo
----------	------------------------	--------

	dominio	pruebas en JS	
5	Regla impuesta por el compilador	Compilación fallida	BUILD FAILED

H. Decisiones de diseño

La Tabla XVII documenta las decisiones arquitectónicas que no se deducen de la lectura del código, junto con la alternativa descartada en cada caso. La Tabla XVIII recoge el entorno de construcción con el que se verificó todo lo anterior.

Una capa, un módulo Gradle	Paquetes dentro de un módulo único	Con paquetes la regla depende de la disciplina; con módulos la impone el compilador
Contenedor de dependencias manual	Hilt / Dagger	Toda la cadena de dependencias queda visible en un archivo; menos complejidad de construcción
Importes en centavos (Long)	Double o Float	El punto flotante binario no representa exactamente los decimales: $0.1 + 0.2 \neq 0.3$
Resultados sellados en vez de excepciones	Propagar excepciones a la interfaz	La UI maneja estados exhaustivos; se elimina una clase de fallos en ejecución
Migración escrita a mano	Recreación destructiva del esquema	Actualizar la aplicación no debe borrar los movimientos del usuario
Reloj inyectado (RelojDominio)	Llamar a now() dentro del caso de uso	La hora es una dependencia del entorno; inyectarla hace deterministas las pruebas y portable el dominio
WorkManager para los recordatorios	AlarmManager con alarma exacta	Avisar dentro de la ventana del día basta; la alarma exacta gasta batería y su permiso está restringido
Signo contable en el tipo, no en el monto	Guardar montos negativos para gastos	Evita errores de cuadre; el monto es siempre positivo y el tipo aporta el signo

Tabla XVIII
ENTORNO DE CONSTRUCCIÓN VERIFICADO

Componente	Versión	Componente	Versión
Kotlin	2.3.21	Room	2.8.4
Android Gradle Plugin	8.7.3	Compose BOM	2025.01.00
Gradle	8.14.5	compileSdk	35
JDK / JVM target	21	minSdk	26
kotlinx-datetime	0.8.0	WorkManager	2.10.5

Cabe registrar una restricción encontrada durante la integración: el procesador de anotaciones de Room no admite los metadatos generados por Kotlin 2.4, lo que obliga a fijar la versión 2.3.21 del lenguaje. El error, `Provided Metadata instance has version 2.4.0, while maximum supported version is 2.3.0`, se reprodujo con las versiones 2.6.1, 2.7.2 y 2.8.4 de Room.

I. Conclusiones

La estructura en tres módulos cumplió los tres objetivos planteados, y uno más que no estaba previsto. El dominio quedó libre de dependencias del framework, lo que permitió que sus 24 pruebas unitarias se ejecuten en la máquina virtual de Java sin emulador, y más tarde que esa misma capa se compilara para JavaScript sin reescribir su lógica. La sustitución del motor de persistencia quedó acotada a un único módulo, ya que ni la presentación ni el dominio nombran a Room. Y la regla de dependencias resultó verificable: la violación deliberada de la Sección VII fue rechazada por el compilador, no por una revisión manual.

El costo de esta organización es real y conviene declararlo: tres módulos exigen más configuración de construcción que uno solo, y una operación sencilla atraviesa más capas que en un diseño monolítico. En un proyecto de esta escala ese costo se paga una sola vez, mientras que el beneficio —poder razonar sobre las reglas de negocio de forma aislada— se cobra en cada cambio posterior.

J. Referencias

XII. APLICACIÓN DE ESTRUCTURAS, ARREGLOS Y CICLOS

Nexcode trabaja con dos estructuras de datos principales. La **lista** (List) representa secuencias ordenadas —los

movimientos de un mes, los pagos fijos del bolsillo, las categorías disponibles— y la **tabla asociativa** (Map) representa búsquedas por clave, sobre todo el índice de categorías por identificador que evita recorrer la lista completa cada vez que una fila de la interfaz necesita saber a qué categoría pertenece.

El recorrido de esas estructuras se resolvió con **funciones de orden superior** en lugar de ciclos for manuales. La razón no es estética: un ciclo manual mezcla en un mismo bloque el recorrido, el filtrado, la acumulación y la construcción del resultado, mientras que la versión declarativa nombra cada operación por separado y hace visible la intención. La Tabla XIX lista las funciones empleadas.

Tabla XIX
FUNCIONES DE ORDEN SUPERIOR UTILIZADAS Y SU PROPÓSITO

Función	Qué hace en el proyecto	Ejemplo de uso
filter	Selecciona un subconjunto	Separar gastos de ingresos
groupBy	Convierte una lista en un mapa por clave	Agrupar movimientos por categoría y por fecha
map	Transforma cada elemento	Entidades de Room a modelos de dominio
sumOf	Acumula un total numérico	Sumar los centavos de una categoría
fold	Acumula con un valor inicial y lógica propia	Saldo neto del día ignorando transferencias
sortedByDescending	Ordena por un criterio	Categorías de mayor a menor gasto
sortedWith	Ordena por varios criterios encadenados	Pagos fijos: activos primero, luego por vencimiento
associateBy	Construye un mapa indexado	Índice de categorías por identificador
count	Cuenta los elementos que cumplen una condición	Pagos vencidos y pagos que vencen hoy
forEach	Ejecuta una acción por elemento	Sembrar los datos iniciales

A. Caso representativo: reparto del gasto por categoría

La función que alimenta el gráfico de dona concentra la mayoría de estas operaciones. Recibe una lista de movimientos y un mapa de categorías, y devuelve una lista ordenada de resúmenes. Nótese que no aparece ningún ciclo explícito:

```

fun summarize(
    transactions: List<Transaction>,
    categoriesById: Map<Long, Category>
): List<CategorySummary> {
    val expenses = transactions.filter { it.type is TransactionType.Expense }
    if (expenses.isEmpty()) return emptyList()

    val grandTotal: Long = expenses.sumOf { it.amount.cents }
    if (grandTotal == 0L) return emptyList()

    return expenses
        .groupBy { it.categoryId } // Map<Long?, List<Transaction>>
        .map { (categoryId, items) ->
            val total = items.sumOf { it.amount.cents }
            CategorySummary(
                category = categoryId?.let { categoriesById[it] }, // safe call
                total = Money(total),
                transactionCount = items.size,
                percentage = total.toFloat() / grandTotal.toFloat()
            )
        }
        .sortedByDescending { it.total.cents }
}

```

Fragmento 1. `domain/usecase/analytics/GetCategorySummaryUseCase.kt` — cinco operaciones encadenadas: filtrar, sumar, agrupar, transformar y ordenar.

La cadena se lee de arriba abajo como una frase: *de los movimientos, tomar los gastos, agruparlos por categoría, convertir cada grupo en un resumen y ordenarlos de mayor a menor*. Un ciclo `for` equivalente necesitaría un acumulador mutable, una condición interna y una estructura auxiliar, y sería más difícil de probar de forma aislada. Esta función es pura —no consulta la base de datos— y por eso tiene cinco pruebas unitarias propias.

```

private fun netTotal(transactions: List<Transaction>): Money =
    transactions.fold(Money.ZERO) { acc, transaction ->
        when (transaction.type) {
            is TransactionType.Transfer -> acc // no altera el saldo
            else -> acc + transaction.signedAmount
        }
    }
}

```

Fragmento 2. `app/presentation/home/HomeScreen.kt` — acumulación con regla de negocio incorporada.

C. El único ciclo explícito del proyecto

Existe un caso en el que un ciclo sí es la herramienta correcta: buscar la próxima fecha en que caerá un día de la semana determinado. El recorrido no es sobre una colección

```

private fun nextWeekly(after: LocalDate, dayOfWeek: Int): LocalDate {
    val target = DayOfWeek.of(dayOfWeek)
    var candidate = after.plusDays(1)
    // Ciclo acotado: como máximo siete intentos.
    repeat(7) {
        if (candidate.dayOfWeek == target) return candidate
        candidate = candidate.plusDays(1)
    }
    return candidate
}

```

Fragmento 3. `domain/model/RecurringFrequency.kt` — ciclo con cota superior demostrable: en siete días siempre aparece cualquier día de la semana.

CRITERIO ADOPTADO

Se usan funciones de orden superior para recorrer colecciones existentes y ciclos explícitos únicamente cuando se generan valores nuevos. En este último caso el ciclo lleva siempre una cota superior demostrable.

B. Acumulación con lógica propia

Cuando la acumulación no es una suma simple se recurre a `fold`, que recibe un valor inicial y decide en cada paso qué hacer. El saldo neto de un día debe ignorar las transferencias entre cuentas propias, porque no alteran el patrimonio total:

existente sino sobre fechas generadas, y termina por una condición. Se acotó deliberadamente con `repeat(7)` para que sea imposible que el ciclo no termine:

Los cuatro pilares de la programación orientada a objetos están presentes en el dominio de Nexcode, aunque Kotlin ofrece para algunos de ellos mecanismos más precisos que los de Java. La Tabla XX resume dónde se materializa cada uno.

Tabla XX
PILARES DE LA POO Y SU MATERIALIZACIÓN EN EL PROYECTO

Pilar	Mecanismo de Kotlin	Clase representativa
Encapsulamiento	Bloque <code>init</code> con <code>require</code> ; propiedades	<code>Transaction · Category</code>

	val	
Abstracción	Interfaces de repositorio	TransactionRepository
Herencia	sealed class con subtipos	TransactionType · NexcodeException
Polimorfismo	Comportamiento propio por subtipo	TransactionType.applySign()
Objetos de valor	value class	Money

A. Encapsulamiento: objetos que no pueden nacer inválidos

En lugar de validar desde fuera antes de construir un objeto, cada modelo protege sus propias invariantes en el constructor. Si alguna condición no se cumple, la instancia no llega a existir:

```
data class Transaction(
    val id: Long = 0L,
    val accountId: Long,
    val categoryId: Long? = null,
    val type: TransactionType,
    val amount: Money,
    val dateTime: LocalDateTime,
    val title: String,
    val note: String? = null
) {
    init {
        require(amount.isPositive) { "El monto debe ser mayor que cero" }
        require(title.isNotBlank()) { "El titulo no puede estar vacio" }
        require(accountId > 0L) { "La transaccion debe pertenecer a una cuenta valida" }
    }

    /** Monto con signo contable: -50.000 para un gasto. */
    val signedAmount: Money get() = type.applySign(amount)
}
```

Fragmento 4. domain/model/Transaction.kt — atributos inmutables, invariantes verificadas en la construcción y un método derivado.

Todos los atributos se declaran `val`, es decir, de solo lectura. Modificar una transacción no muta el objeto sino que produce uno nuevo mediante `copy()`, lo que elimina toda una familia de errores por estado compartido. Una decisión de diseño relacionada es que **el monto es siempre positivo y el signo contable lo aporta el tipo**: guardar montos negativos para los gastos es una fuente clásica de errores de cuadre.

El tipo de movimiento es una jerarquía de herencia: una clase base abstracta con tres subclases. Al declararla `sealed`, el compilador conoce la lista completa de subtipos, de modo que una expresión `when` sobre ella es exhaustiva y no necesita `else`. Si mañana se agregara un cuarto tipo de movimiento, el compilador señalaría todos los lugares donde falta contemplarlo.

B. Herencia y polimorfismo mediante jerarquías selladas

```
sealed class TransactionType(
    val code: String,
    val sign: Int
) {
    data object Income : TransactionType(code = "INCOME", sign = +1)
    data object Expense : TransactionType(code = "EXPENSE", sign = -1)
    data class Transfer(val targetAccountId: Long)
        : TransactionType(code = "TRANSFER", sign = 0)

    /** Aplica el signo contable al monto. */
    fun applySign(amount: Money): Money = Money(amount.cents * sign)

    fun defaultTitle(): String = when (this) {
        is Income -> "Ingreso"
        is Expense -> "Gasto"
        is Transfer -> "Transferencia"
    } // sin `else`: la sealed class garantiza exhaustividad
}
```

Fragmento 5. domain/model/TransactionType.kt — herencia sellada. *Income* y *Expense* son objetos únicos porque no llevan datos propios; *Transfer* es una clase porque necesita la cuenta destino.

El polimorfismo se manifiesta en que `applySign` produce un resultado distinto según el subtipo sin necesidad de condicionales: cada subclase aporta su propio `sign` al

constructor de la clase base. La Fig. 9 muestra las dos jerarquías selladas del proyecto.

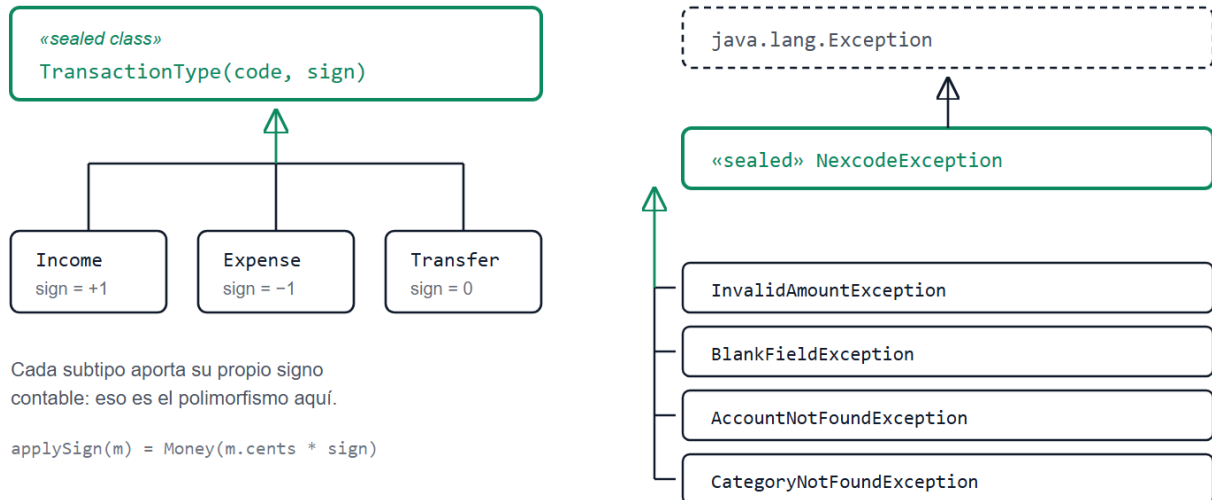


Fig. 9. Las dos jerarquías de herencia del dominio. A la izquierda, `TransactionType`, cuyos subtipos aportan el signo contable. A la derecha, `NexcodeException`, que hereda de `Exception` y agrupa los cuatro errores de negocio; al ser sellada, el conjunto de errores posibles del dominio es cerrado y conocido en compilación.

C. Objetos de valor: el dinero como tipo propio

Un importe no es un número cualquiera: no tiene sentido multiplicar dos importes entre sí, y sumar pesos con centavos por descuido produce errores silenciosos. Por eso el dinero se

modela como una clase propia que envuelve un entero de centavos y expone únicamente las operaciones que tienen sentido, mediante sobrecarga de operadores:

```

@JvmInline
value class Money(val cents: Long) : Comparable<Money> {
    val isPositive: Boolean get() = cents > 0

    operator fun plus(other: Money): Money = Money(cents + other.cents)
    operator fun minus(other: Money): Money = Money(cents - other.cents)
    operator fun unaryMinus(): Money = Money(-cents)

    override fun compareTo(other: Money): Int = cents.compareTo(other.cents)

    /** Formato colombiano en pesos enteros: 1.250.000 */
    fun format(withSign: Boolean = false): String { /* ... */ }
}

```

Fragmento 6. `domain/model/Money.kt` — el modificador `value` hace que el compilador reemplace la clase por un `Long` en tiempo de ejecución: se obtiene seguridad de tipos sin costo de memoria.

XIV. MANEJO DE EXCEPCIONES

La estrategia adoptada parte de una regla: **las excepciones no cruzan hacia la capa de presentación**. Una pantalla no

debe envolver llamadas en bloques `try`; debe recibir un estado y pintarlo. Para lograrlo, el manejo de errores se organiza en tres niveles sucesivos.

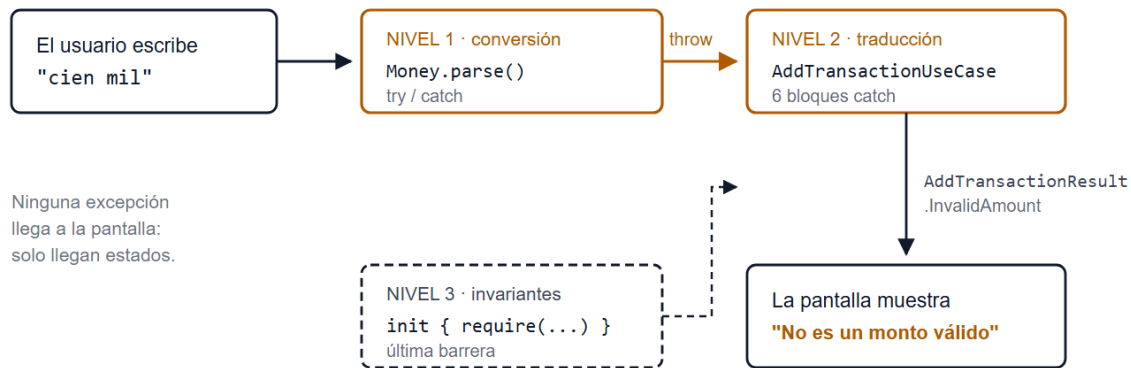


Fig. 10. Recorrido de un error desde el teclado hasta el mensaje en pantalla. La excepción existe únicamente entre los niveles 1 y 2; a partir de ahí el error viaja como un valor de un tipo sellado. El nivel 3 son las invariantes del modelo, que actúan si algo llegara a escapar de las validaciones anteriores.

A. Nivel 1: conversión de texto a número

El punto más frágil de la aplicación es la conversión del texto que escribe el usuario a un valor numérico. Se resuelve

```

return try {
    Money(
        BigDecimal(normalized)
            .movePointRight(2)
            .setScale(0, RoundingMode.HALF_UP)
            .toLong()
    )
} catch (e: NumberFormatException) {
    throw InvalidAmountException("\\" + row + "\" no es un monto valido", e)
} catch (e: ArithmeticException) {
    throw InvalidAmountException("El monto\\" + row + "\" excede el limite permitido", e)
}
    
```

Fragmento 7. domain/model/Money.kt — la excepción original viaja como cause, de modo que no se pierde información para la depuración.

B. Nivel 2: traducción a resultados

El caso de uso envuelve toda su lógica en un único bloque try y declara seis capturas ordenadas de la más específica a la

```

// MANEJO DE EXCEPCIONES: de la mas especifica a la mas general.
} catch (e: InvalidAmountException) {
    AddTransactionResult.InvalidAmount(e.message ?: "Monto invalido")
} catch (e: BlankFieldException) {
    AddTransactionResult.MissingField(e.fieldName)
} catch (e: AccountNotFoundException) {
    AddTransactionResult.NotFound(e.message ?: "Cuenta no encontrada")
} catch (e: CategoryNotFoundException) {
    AddTransactionResult.NotFound(e.message ?: "Categoria no encontrada")
} catch (e: IllegalArgumentException) {
    // Disparada por los require() del modelo de dominio.
    AddTransactionResult.MissingField(e.message ?: "Datos incompletos")
} catch (e: Exception) {
    // Red de seguridad: fallo de Room, disco lleno, etc.
    AddTransactionResult.Failure(
        "No se pudo guardar la transaccion: " + (e.message ?: "error desconocido")
    )
}
    
```

Fragmento 8. domain/usecase/transaction/AddTransactionUseCase.kt — el orden importa: IllegalArgumentException va antes que Exception, y la captura general queda de última como red de seguridad.

La Tabla XXI relaciona cada situación de error con lo que finalmente ve el usuario.

con un try que captura las dos excepciones que la biblioteca estándar puede lanzar y las convierte en una excepción propia del dominio, conservando la causa original:

más general. Cada una produce una variante distinta del resultado:

Tabla XXI
CORRESPONDENCIA ENTRE ERRORES, RESULTADOS Y MENSAJES

Situación	Excepción	Resultado	Mensaje en pantalla
Campo de monto vacío	InvalidAmountException	InvalidAmount	El monto no puede estar vacío
Texto no numérico	InvalidAmountException	InvalidAmount	«cien mil» no es un monto válido
Monto negativo o cero	InvalidAmountException	InvalidAmount	El monto debe ser mayor que cero
Cuenta inexistente	AccountNotFoundException	NotFound	No existe la cuenta con id N
Transferencia a la misma cuenta	BlankFieldException	MissingField	Falta un dato: cuenta destino
Fallo de la base de datos	Exception	Failure	No se pudo guardar la transacción

VERIFICACIÓN EN DISPOSITIVO

El camino completo se comprobó en el emulador: al pulsar «Guardar» con el campo de monto vacío, el volcado de la interfaz mostró el nodo de texto "El monto no puede estar vacío", confirmando que el mensaje nace en el dominio y llega intacto a la pantalla sin que ninguna excepción se propague.

XV. SEGURIDAD CONTRA VALORES NULOS

Kotlin distingue en el sistema de tipos entre una referencia que puede ser nula (`String?`) y una que no (`String`). El compilador impide llamar a un método sobre la primera sin haber comprobado antes su contenido, lo que elimina en tiempo de compilación la excepción de puntero nulo.

En Nexcode la nulabilidad no se usa por comodidad sino que **modela hechos del negocio**. Un movimiento puede no tener categoría, porque la política `ON DELETE SET NULL` del esquema deja huérfanos los movimientos cuando se borra una categoría; una nota es opcional porque el usuario no está obligado a escribirla; un presupuesto mensual puede no existir. La Tabla XXII recoge los campos opcionales y su justificación.

Tabla XXII
CAMPOS OPCIONALES DEL DOMINIO Y SU RAZÓN DE SER

Campo	Tipo	Por qué puede ser nulo
Transaction.categoryId	Long?	La base anula la clave al borrar la categoría
Transaction.note	String?	La nota es opcional para el usuario
Category.parentId	Long?	Nulo significa «categoría raíz»
Category.monthlyBudget	Money?	No toda categoría tiene presupuesto
RecurringPayment.lastPaidDate	LocalDate?	Un pago nuevo aún no se ha pagado nunca
AdviceRequest.userQuestion	String?	El usuario puede pedir análisis sin preguntar nada

```
// 2. Cuenta: Elvis con throw como valor de la expresion.
val account = accountRepository.findById(input.accountId)
?: throw AccountNotFoundException(input.accountId)

// 3. Categoría: safe call. Si categoryId es null NO se consulta la base.
val category: Category? = input.categoryId?.let { id ->
    categoryRepository.findById(id) ?: throw CategoryNotFoundException(id)
}

// 4. Título: cadena de Elvis con tres niveles de respaldo.
val title: String = input.rawTitle?.trim()?.takeIf { it.isNotBlank() }
?: category?.name
?: input.type.defaultTitle()
```

A. Operadores empleados

La Tabla XXIII resume los operadores de seguridad ante nulos utilizados, con el número de apariciones contadas sobre la capa de dominio.

Tabla XXIII
OPERADORES DE SEGURIDAD ANTE NULOS EN LA CAPA DE DOMINIO

Operador	Función	Apariciones
?:	Elvis: valor alternativo cuando la expresión es nula	42
?.trim()	Llamada segura: solo se ejecuta si no es nulo	9
takeIf { }	Convierte en nulo lo que no cumple la condición	8
?.let { }	Ejecuta un bloque solo si hay valor	6
orEmpty()	Convierte una cadena nula en cadena vacía	2
isNullOrBlank()	Comprueba nulo y vacío en una sola llamada	1
!!	Aserción no nula (descartada por política)	0

POLÍTICA DEL PROYECTO

El operador `!!` no se usa en ninguna de las 6 548 líneas del proyecto. Ese operador desactiva la comprobación del compilador y reintroduce exactamente el fallo que el sistema de tipos evita; cada caso en que fue tentador se resolvió con Elvis o con una llamada segura.

B. Caso representativo: cadena de respaldos

El fragmento más ilustrativo está en el caso de uso de registro. El título del movimiento se resuelve con una cadena de tres niveles: lo que escribió el usuario, si no está en blanco; en su defecto el nombre de la categoría; y si tampoco hay categoría, el título por defecto del tipo de movimiento. La expresión no puede producir nulo:

```
// 5. Nota opcional: " " se normaliza a null, no a cadena vacía.
val note: String? = input.rawNote?.trim()?.takeIf { it.isNotBlank() }
```

Fragmento 9. `domain/usecase/transaction/AddTransactionUseCase.kt` — cuatro usos distintos de la seguridad ante nulos en doce líneas.

Tres detalles merecen comentario. Primero, `?.trim()?.takeIf { it.isNotBlank() }` aprovecha que en Kotlin `throw` es una expresión, lo que permite obtener un valor no nulo o interrumpir en una sola línea. Segundo, la llamada segura sobre `categoryId` evita una consulta innecesaria a la base de datos cuando no hay categoría. Tercero, la combinación `?.trim()?.takeIf { it.isNotBlank() }` normaliza a nulo una nota compuesta solo de espacios, de modo que la base de datos nunca almacena cadenas en blanco que después obliguen a comprobar dos condiciones distintas.

Del lado de la lectura, los modelos exponen propiedades derivadas que entregan a la interfaz un valor siempre seguro,

como `val displayNote: String get() = note ?: "Sin nota"`. La pantalla no comprueba nulos: recibe texto listo para pintar.

XVI. PROCESO DE DEPURACIÓN

Durante el desarrollo se registraron cinco fallos representativos, documentados en la Tabla XXIV con su síntoma, causa raíz y solución. Se incluyen deliberadamente tanto errores de configuración como errores de lógica y un falso positivo, porque el proceso de depuración incluye descartar hipótesis equivocadas.

Tabla XXIV
REGISTRO DE DEPURACIÓN

N.º	Síntoma	Causa raíz	Solución	Detectado por
1	Falla el procesador de anotaciones de Room	Room no admite los metadatos de Kotlin 2.4	Fijar Kotlin 2.3.21 con Room 2.8.4	Compilador
2	Cannot access 'RoomDatabase'	La capa de presentación tocaba Room directamente	Encapsular el ensamblaje en <code>DataProvider</code>	Compilador
3	«25.000» se guardaba como 25 pesos	Regla de parseo ambigua con el punto	Regla del separador de miles por tres dígitos	Prueba unitaria
4	Falla la compilación en el análisis estático	<code>windowLightNavigationBar</code> exige API 27	Separar el atributo en <code>values-v27/</code>	Lint
5	La app «se cerraba» al deslizar	Falso positivo: gesto «atrás» del sistema	Repetir la prueba lejos del borde	Inspección visual

A. Fallo 3: el error de lógica que atrapó una prueba

Es el caso más instructivo porque no produjo ningún mensaje de error: la aplicación funcionaba y guardaba un valor incorrecto. Al escribir 25.000 —veinticinco mil pesos en notación colombiana— el sistema almacenaba 25 pesos,

```
val lastComma = text.lastIndexOf(',')
val lastDot = text.lastIndexOf('.')
val digitsAfterLastDot = if (lastDot >= 0) text.length - lastDot - 1 else -1

val normalized = when {
    // Hay coma: la coma manda como decimal y los puntos son miles.
    lastComma >= 0 -> text.replace(".", "").replace(',', '.')
    // Solo puntos, el ultimo con 3 digitos detras: separador de miles.
    lastDot >= 0 && digitsAfterLastDot == 3 -> text.replace(".", "")
    // Resto de casos: el punto es el separador decimal.
    else -> text
}
```

Fragmento 10. `domain/model/Money.kt` — regla de desambiguación entre separador decimal y separador de miles.

Junto con la corrección se incorporó una prueba de regresión que fija el comportamiento para el futuro:

```
@Test
fun `parse trata el punto con tres digitos como separador de miles`() {
    assertEquals(2_500_000L, Money.parse("25.000").cents)
}
```

Fragmento 11. `domain/test/MoneyTest.kt` — la prueba documenta la regla y evita que el fallo regrese.

B. Fallo 2: el compilador como detector de fallas de arquitectura

```
e: AppContainer.kt:45:44 Cannot access 'androidx.room.RoomDatabase'
which is a supertype of 'NexcodeDatabase'.
Check your module classpath for missing or conflicting dependencies.
```

porque la regla original interpretaba un punto único como separador decimal.

El fallo se detectó al escribir la prueba unitaria del caso de uso de registro, antes de que ningún usuario tocara la aplicación. La corrección consistió en formalizar la regla de notación:

Al conectar la interfaz con la base de datos, el módulo de presentación dejó de compilar con un mensaje que a primera vista parecía un problema de dependencias:

El diagnóstico correcto no era agregar Room como dependencia de la capa de presentación —eso habría hecho desaparecer el error y, con él, la separación de capas— sino reconocer que el mensaje señalaba una fuga arquitectónica: la presentación estaba construyendo la base de datos. La solución fue mover ese ensamblaje al interior de la capa de datos, tras la fábrica `DataProvider`. El error desapareció y la arquitectura quedó más estricta que antes.

C. Fallo 5: descartar un falso positivo

Al probar el borrado por deslizamiento mediante automatización, la aplicación desaparecía de la pantalla y

quedaba visible otra aplicación. La hipótesis inicial fue un fallo grave en el componente de deslizamiento. La inspección de la captura mostró que la aplicación visible era otro proyecto del mismo equipo, lo que descartaba un cierre por error: el gesto de prueba, iniciado en el borde derecho de la pantalla, había activado el gesto «atrás» del sistema operativo. Al repetir el mismo gesto lejos del borde, el borrado funcionó correctamente. El fallo estaba en el método de prueba, no en el código.

D. Herramientas empleadas

Tabla XXV
HERRAMIENTAS DE DEPURACIÓN UTILIZADAS

Herramienta	Uso en el proyecto
Compilador de Kotlin	Detección de incompatibilidades y de violaciones de capa
Pruebas unitarias (JUnit)	19 pruebas del dominio; detectaron el fallo 3
Lint de Android	Análisis estático; detectó el fallo 4
Logcat	Verificación de ausencia de excepciones fatales tras cada instalación
uiautomator dump	Comprobación del texto real en pantalla, sin depender de la vista
Capturas del emulador	Verificación visual del comportamiento y descarte del fallo 5

OBSERVACIÓN

Cuatro de los cinco fallos fueron detectados por herramientas automáticas —compilador, pruebas y análisis estático— antes de ejecutar la aplicación. El único que exigió inspección manual resultó ser un falso positivo. Esto respalda una decisión tomada al inicio del proyecto: invertir en pruebas del dominio que corren en la máquina virtual de Java, porque su ciclo de retroalimentación se mide en segundos y no en minutos de emulador.

XVII. IMPLEMENTACIÓN MULTIPLATAFORMA

La Sección 11 sostiene que la capa de dominio está aislada del framework. Esa afirmación se apoyaba en una prueba negativa: no aparecen imports de Android en sus archivos. Es una evidencia razonable, pero indirecta; un módulo puede estar libre de Android y seguir atado a la plataforma por la vía de la biblioteca estándar de Java, que en Android está disponible y por tanto no produce ningún error.

La prueba concluyente de que una capa es independiente de la plataforma consiste en **compilarla para otra**. Con esa finalidad se migró el módulo `:domain` a Kotlin Multiplatform.

A. Qué significa multiplataforma en este proyecto

Kotlin Multiplatform permite escribir código una sola vez en un conjunto de fuentes común y compilarlo para varios destinos. El conjunto `commonMain` no tiene acceso a la biblioteca de Java ni a las API de ninguna plataforma concreta: solo a la biblioteca estándar de Kotlin y a bibliotecas que a su vez sean multiplataforma. Esa restricción no la impone una convención del equipo, la impone el compilador.

Se eligieron dos objetivos. **JVM**, porque es el que consumen el módulo `:data` y la aplicación Android. Y **JavaScript** sobre Node.js, porque es una plataforma genuinamente distinta —no comparte biblioteca estándar con Java— y por tanto sirve como verificación real de portabilidad. La Fig. 11 muestra la estructura resultante.



Fig. 11. Estructura de conjuntos de fuentes tras la migración. Todo el dominio vive en commonMain y se compila hacia dos destinos. El destino JavaScript no tiene consumidor en producción: su función es verificar que la lógica de negocio no depende de la plataforma.

B. Qué hubo que eliminar del dominio

Antes de la migración el dominio contenía dieciséis referencias a la biblioteca de Java, ninguna de las cuales

producía error alguno compilando para Android. La Tabla XXVI enumera cada dependencia y su sustituto.

Tabla XXVI
DEPENDENCIAS DE JAVA RETIRADAS DEL DOMINIO

Dependencia	Uso que tenía	Sustituto
java.math.BigDecimal	Convertir el texto del usuario a centavos con redondeo exacto	Aritmética entera propia sobre cadenas
java.time.LocalDatejava.time.LocalDateTime	Fechas de movimientos y vencimientos	kotlinx-datetime 0.8.0
java.time.DayOfWeekjava.time.temporal.ChronoUnit	Periodicidad semanal y días hasta el vencimiento	kotlinx.datetime.DayOfWeek y toEpochDays()
java.time.YearMonth	Filtrar los movimientos del mes en curso	Clase propia MesDelAnio
java.time.Clock	Obtener la fecha y hora actuales	Interfaz inyectable RelojDominio
java.util.SortedMap(via toSortedMap)	Ordenar por fecha los movimientos agrupados	sortedByDescending + associate

La sustitución del reloj merece un comentario aparte porque no fue solo un cambio de biblioteca. El caso de uso de registro llamaba directamente a `LocalDateTime.now()`, lo que ataba el dominio a la plataforma y hacía imposible escribir una

prueba que verificara la fecha guardada. Al reemplazarlo por una interfaz inyectable de una sola función, ambos problemas desaparecieron a la vez:

```

// commonMain – el dominio declara lo que necesita, no cómo se obtiene
fun interface RelojDominio {
    fun ahora(): LocalDateTime
}

// :app – la plataforma cumple el contrato
class RelojSistema : RelojDominio {
    override fun ahora(): LocalDateTime =
        java.time.LocalDateTime.now().toKotlinLocalDateTime()
}

// commonTest – la prueba fija el tiempo
val RELOJ_FIJO = RelojDominio { LocalDateTime(2026, 8, 22, 12, 0) }
  
```

Fragmento 12. El mismo patrón de inversión de dependencias de la Sección 11, aplicado ahora al tiempo: el dominio declara el contrato y cada plataforma lo cumple.

C. Los acoplamientos que solo el compilador de JavaScript reveló

Este es el resultado más útil de la migración. Tras retirar las dieciséis referencias de la Tabla XXVI, el módulo compilaba sin errores para JVM. Se habría podido concluir que el trabajo

estaba terminado. Sin embargo, al compilar para JavaScript aparecieron dos errores más:

```
$ ./gradlew :domain:compileKotlinJs

e: Money.kt:16:2  Unresolved reference 'JvmInline'.
e: GetTransactionsUseCase.kt:29:18  Unresolved reference 'toSortedMap'.

BUILD FAILED
```

El primero es sutil: `@JvmInline` es obligatorio para declarar una *value class*, y en un módulo JVM el compilador lo resuelve automáticamente. En código común no: exige el `import` explícito `kotlin.jvm.JvmInline`, porque es una anotación de expectativa opcional que cada plataforma implementa a su manera.

El segundo es más grave desde el punto de vista arquitectónico. La función `toSortedMap` parece Kotlin

idiomático —se encadena igual que `map` o `filter`— pero pertenece a `java.util.SortedMap`. Estaba en medio de una cadena de operaciones de colecciones, sin ningún `import` que la delatara, y ninguna revisión manual la había detectado. La corrección fue reescribir la operación con funciones de la biblioteca estándar de Kotlin:

```
// Antes: toSortedMap viene de java.util, invisible en la cadena
transactions
    .groupBy { it.day }
    .toSortedMap(compareByDescending { it })
    .mapValues { entry -> entry.value.sortedByDescending { it.dateTime } }

// Después: solo biblioteca estándar de Kotlin
transactions
    .groupBy { it.day }
    .toList()
    .sortedByDescending { (dia, _) -> dia }
    .associate { (dia, movimientos) ->
        dia to movimientos.sortedByDescending { it.dateTime }
    }
```

Fragmento 13. `domain/usecase/transaction/GetTransactionsUseCase.kt` — la versión portable produce el mismo resultado: *associate* conserva el orden de inserción.

HALLAZGO
Dos acoplamientos a la plataforma sobrevivieron a la revisión manual y a la compilación para JVM, y solo se hicieron visibles al compilar para JavaScript. Esto sustenta la tesis de la sección: **la independencia de plataforma no se declara, se comprueba compilando para otra**. Un módulo que solo se compila para su destino habitual puede estar acoplado sin que nadie lo note.

D. Dónde ocurre la traducción entre mundos
El dominio dejó de hablar `java.time`, pero las capas que lo rodean sí lo necesitan: Room almacena las fechas, y `DateTimeFormatter` es lo que sabe escribir «Sábado 22 de agosto» en español. La conversión entre ambos sistemas de tipos se concentró deliberadamente en **cinco archivos frontera**, listados en la Tabla XXVII. Fuera de ellos, ningún archivo del proyecto mezcla los dos mundos.

Tabla XXVII
ARCHIVOS FRONTERA DONDE SE TRADUCE ENTRE KOTLINX-DATETIME Y JAVA.TIME

Archivo	Capa	Motivo de la traducción
RoomConverters.kt	Datos	SQLite guarda números y texto, no fechas
TransactionRepositoryImpl.kt	Datos	Convertir un rango de días a milisegundos para la consulta
DefaultData.kt	Datos	Fechar los datos de demostración respecto de hoy
RelojSistema.kt	Presentación	Cumplir el contrato RelojDominio con el reloj del dispositivo
Formato.kt	Presentación	Escribir meses y días en español con <code>DateTimeFormatter</code>

E. Qué se comparte y qué no puede compartirse

Conviene delimitar el alcance con precisión, porque «multiplataforma» admite lecturas más amplias que la que aplica aquí. Lo compartido es **la capa de dominio completa**: modelos, reglas de validación, casos de uso, cálculos y contratos. Lo que no se comparte es todo aquello que por definición depende del sistema operativo, y la Tabla XXVIII lo enumera sin eufemismos.

Tabla XXVIII

ALCANCE REAL DE LO COMPARTIDO		
Componente	¿Compartido?	Razón
Modelos y reglas de negocio	Sí	Son cálculo puro
Los 17 casos de uso	Sí	Solo dependen de los contratos
Contratos de repositorio	Sí	Son interfaces
Las 24 pruebas unitarias	Sí	Viven en <code>commonTest</code>
Persistencia (Room / SQLite)	No	Room es una biblioteca de Android
Interfaz (Jetpack)	No	Vinculada al sistema

Compose)		de vistas de Android
Notificaciones y WorkManager	No	El trabajo en segundo plano es propio de cada sistema

La prueba de que la migración cumplió su objetivo es que **el mismo archivo de pruebas se ejecuta en las dos plataformas y produce el mismo resultado**. No se trata de dos suites paralelas: es un único conjunto en `commonTest`, compilado dos veces.

F. Verificación

```
$ ./gradlew :domain:jvmTest :domain:jsNodeTest

jvmTest      · MoneyTest ..... 12 pruebas, 0 fallos
jvmTest      · AddTransactionUseCaseTest ..... 7 pruebas, 0 fallos
jvmTest      · GetCategorySummaryUseCaseTest .. 5 pruebas, 0 fallos

jsNodeTest   · MoneyTest ..... 12 pruebas, 0 fallos
jsNodeTest   · AddTransactionUseCaseTest ..... 7 pruebas, 0 fallos
jsNodeTest   · GetCategorySummaryUseCaseTest .. 5 pruebas, 0 fallos

BUILD SUCCESSFUL
```

Que las pruebas de Money pasen en JavaScript tiene un valor añadido: el análisis de montos se reescribió sin `BigDecimal`, y JavaScript no tiene enteros de 64 bits nativos — Kotlin los emula—. Que el redondeo al centavo produzca el mismo resultado en ambas plataformas confirma que la reimplementación es correcta y no solo compilable.

G. Limitaciones y trabajo futuro

El alcance alcanzado es real pero acotado, y conviene declararlo con la misma claridad con que se declara el logro.

No hay objetivo iOS. Compilar para iOS exige un equipo con macOS y las herramientas de Xcode, de las que el equipo no dispone. La restricción es de infraestructura, no de diseño: el código de `commonMain` no usa nada que impida añadir ese objetivo, y hacerlo consistiría en declararlo en el archivo de construcción.

La interfaz no se comparte. Nexcode tiene una sola interfaz, escrita en Jetpack Compose para Android. Compartirla exigiría migrar a Compose Multiplatform, que es un proyecto de otra magnitud y no formaba parte de los objetivos.

El objetivo JavaScript no tiene consumidor. Existe como verificación, no como producto: no hay una versión web de Nexcode. Su valor es el descrito en el apartado C, y ese valor se cobra en cada compilación futura, porque cualquier acoplamiento nuevo a la plataforma romperá el build.

COSTO ASUMIDO

La migración tuvo un precio que conviene registrar: el dominio ya no usa `BigDecimal`, de modo que el análisis de montos es código propio y no una biblioteca probada por terceros. Se compensó con ocho pruebas unitarias específicas de Money, incluidas las de redondeo del tercer decimal y de límite superior. Es un intercambio consciente: se cambió una dependencia de plataforma por código propio bajo prueba.

H. Conclusión

La migración confirmó la afirmación de la Sección 11 y, a la vez, mostró que la evidencia en que esa afirmación se apoyaba era insuficiente. Contar imports de Android detecta el acoplamiento evidente; compilar para otra plataforma detecta el acoplamiento silencioso. Los dos errores del apartado C llevaban semanas en el repositorio sin que ninguna herramienta anterior los señalara.

El resultado práctico es que la lógica de negocio de Nexcode —qué hace válido un gasto, cómo se reparte el gasto por categoría, cuándo vence un pago fijo— es hoy código portable, verificado en dos plataformas, que podría reutilizarse en una versión web o de escritorio sin reescribir una línea.

XVIII. INTEGRACIÓN O ASISTENCIA MEDIANTE IA

Nexcode incorpora un asistente financiero que interpreta los datos del usuario y devuelve una lectura en lenguaje natural, y que admite **conversación por voz**: el usuario pregunta hablando y la aplicación responde en voz alta. Su diseño responde a una pregunta previa: *¿qué parte de un asistente es lógica de negocio y qué parte es infraestructura?* La respuesta adoptada es que **decidir qué información merece un consejo** pertenece al dominio, mientras que **quién redacta ese consejo** es un detalle sustituible.

Por eso el dominio declara únicamente un contrato, `FinancialAdvisor`, con una sola función. Quien lo cumpla — un motor de reglas local o un agente conversacional remoto— es una decisión de la capa de datos que el resto de la aplicación desconoce. Es el mismo patrón de inversión de dependencias que la Sección 11 aplica a la persistencia.

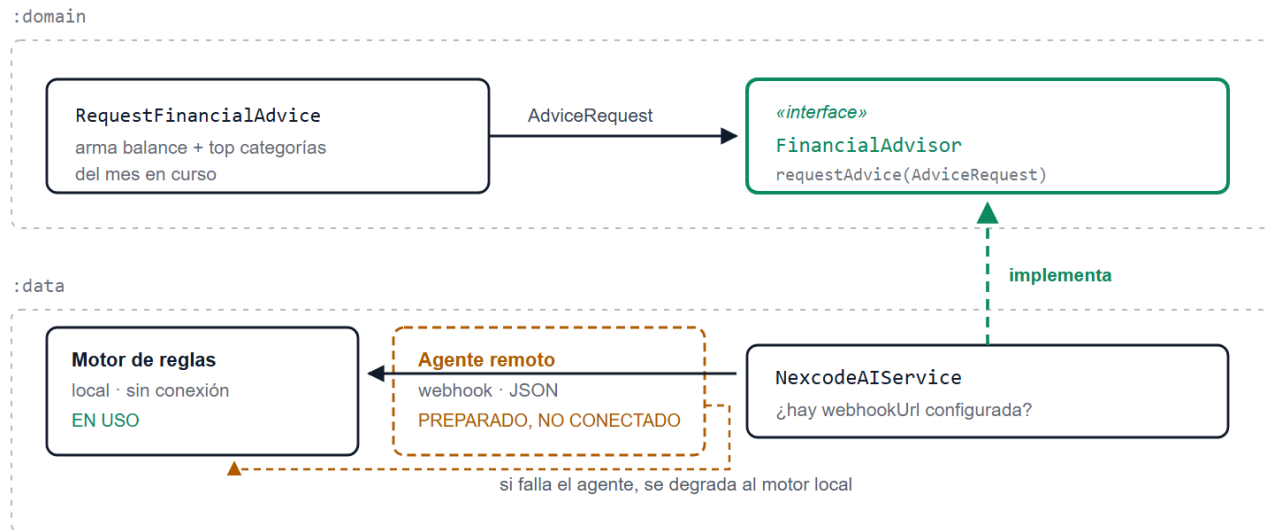


Fig. 12. Arquitectura del asistente. El dominio solo conoce la interfaz; la implementación decide entre el motor local y el agente remoto según haya o no una dirección de webhook configurada. Si el agente remoto falla, la respuesta se degrada al motor local en lugar de mostrar un error.

A. Lo que el dominio entrega al asistente

La petición es un modelo de dominio, no una estructura de red: contiene el periodo, el balance, las cinco categorías de mayor gasto y, opcionalmente, la pregunta del usuario. Nótese

que `userQuestion` es un tipo opcional porque el usuario puede pedir un análisis sin preguntar nada concreto, lo que enlaza con la seguridad ante nulos de la Sección 15.

```

data class AdviceRequest(
    val periodLabel: String,
    val balance: Balance,
    val topCategories: List<CategorySummary>,
    val userQuestion: String? = null // null = análisis sin pregunta
)

data class AdviceResponse(
    val message: String,
    val tips: List<String> = emptyList(),
    val source: Source = Source.LOCAL_SIMULATION
) {
    enum class Source { LOCAL_SIMULATION, REMOTE_AGENT }
}
  
```

Fragmento 14. `domain/ai/FinancialAdvisor.kt` — la respuesta declara su origen, de modo que la interfaz puede indicar al usuario si el análisis es local o remoto.

B. Motor de reglas local

La implementación que opera hoy analiza los datos con un conjunto de reglas sobre la tasa de ahorro y la concentración

del gasto. No es un modelo de lenguaje: es una función determinista que produce el mismo consejo ante los mismos datos, lo que además la hace verificable.

Tabla XXIX
REGLAS DEL MOTOR LOCAL

Condición	Consejo generado
Sin ingresos registrados	Sugiere registrar el salario para poder medir la capacidad de ahorro
Tasa de ahorro negativa	Advierte que el gasto supera al ingreso y señala recortar la categoría mayor
Ahorro inferior al 10 %	Indica la meta sana de entre 15 % y 20 % del ingreso
Ahorro igual o superior al 20 %	Confirma que la situación es buena
Una categoría supera el 40 % del gasto	Señala esa categoría como la mayor palanca de ahorro
Una categoría con 10 o más movimientos	Advierte sobre gastos pequeños que se acumulan

La salida obtenida sobre datos reales durante la verificación fue la siguiente:

Analisis local (sin conexion)

Este mes ingresaste 0 y gastaste 25.000. Tu saldo es -25.000.

- 1. Aun no registras ingresos este mes: agrega tu salario para medir tu capacidad de ahorro.
- 2. Comida concentra el 100% de tus gastos. Ahi esta tu mayor palanca de ahorro.

Salida 1. Respuesta del asistente durante el caso de prueba S-12, generada a partir de los movimientos reales del usuario.

C. Asistente de voz con las API del sistema

El enunciado del proyecto pedía un asistente de voz. La vía habitual sería contratar una plataforma de agentes conversacionales, pero Android incluye de fábrica las dos mitades necesarias: SpeechRecognizer [31] convierte voz en texto y TextToSpeech [32] convierte texto en voz. Usarlas evita el costo por minuto, elimina la dependencia de un servicio

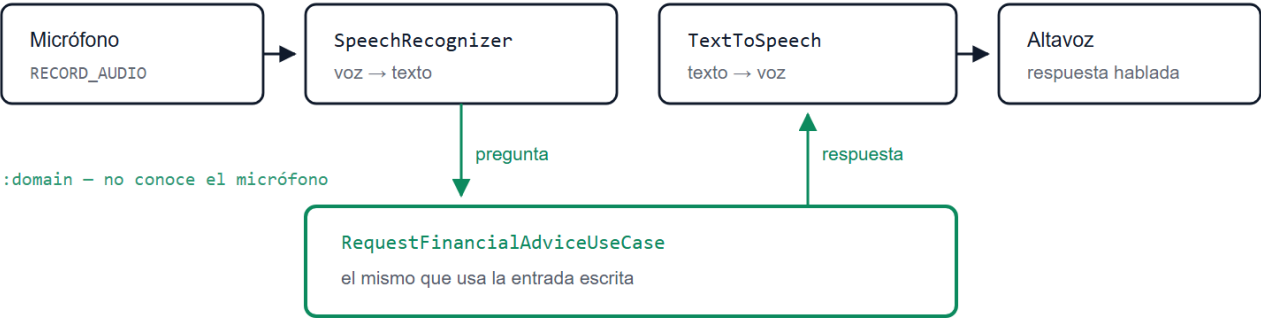
externo y respeta la restricción R-1 de la Sección 7, porque en los dispositivos que tienen instalado el paquete de idioma ambas funcionan sin conexión.

La decisión, además, no obligó a modificar la lógica de negocio. La voz es simplemente **otra entrada y otra salida** sobre el mismo contrato FinancialAdvisor: el dominio no contiene una sola referencia al micrófono.

Tabla XXX
OPCIONES EVALUADAS PARA EL ASISTENTE DE VOZ

Opción	Costo	Sin conexión	Motivo de la decisión
API del sistema Android	Ninguno	Sí, con el paquete de idioma	Elegida. Cubre el requisito sin servicio externo ni cuenta
Agente de voz comercial	Por minuto hablado	No	Descartada: exige sesión de audio en tiempo real y presupuesto recurrente
Modelo de lenguaje por API	Por consulta	No	Complementaria: mejoraría el texto, no aporta la voz

CAPA DE PRESENTACIÓN — depende del dispositivo



Cambiar la voz por texto solo cambia las cajas de arriba: el centro no se entera.

Fig. 13. Recorrido de una consulta por voz. Las cajas superiores pertenecen a la capa de presentación y dependen del dispositivo; el caso de uso del centro es exactamente el mismo que atiende la entrada escrita. Es la separación de capas de la Sección 11 aplicada a una modalidad de interacción nueva.

Tres decisiones de comportamiento merecen registro. La aplicación **solo responde hablando si la pregunta llegó hablada**: si el usuario escribe y pulsa el botón, la respuesta aparece en silencio, porque un asistente que habla sin que se lo pidan resulta molesto. La lectura usa QUEUE_FLUSH y no QUEUE_ADD, de modo que un análisis nuevo interrumpe al anterior en lugar de encolarse detrás de una respuesta ya obsoleta. Y los dos motores se liberan en el DisposableEffect

de la pantalla, porque retienen recursos del sistema: no hacerlo dejaría el micrófono ocupado después de salir.

D. Un fallo del reconocimiento y su corrección

La primera versión solicitaba el motor local con EXTRA_PREFER_OFFLINE = true, coherente con el requisito de funcionar sin conexión. En el dispositivo de prueba el reconocimiento falló siempre, y el registro del sistema mostró la causa:

```
W ReconocedorDeVoz: Error de reconocimiento: codigo 12
E SodaSpeechRecognizer: Failed to get language pack of required locale: error 12
W RecognitionServiceImpl: Speech recognition error type LANGUAGE_PACK_ERROR
```

El dispositivo no tenía instalado el paquete de español para reconocimiento sin conexión. La corrección no consistió en renunciar al modo local —se perdería el funcionamiento sin datos en los equipos que sí lo tienen— sino en **reintentar una**

sola vez con el motor en línea, salvo para los errores en que reintentar no aporta nada: que el usuario no dijo nada, que se agotó el tiempo de espera, que falta el permiso o que el motor está ocupado.

```
override fun onError(error: Int) {
    // Un fallo del motor local no es definitivo: puede faltar el
    // paquete de idioma sin conexión. Se reintentará una vez en línea.
```

```

val vuelveAFallar = error in ERRORES_SIN_REINTENTO

if (preferirSinConexion && !vuelveAFallar) {
    escuchar(onParcial, onResultado, onError, preferirSinConexion = false)
} else {
    onError(mensajeDeError(error))
}
}

```

Fragmento 15. *presentation/voz/ReconocedorDeVoz.kt* — tras la corrección, el registro muestra el error 12 seguido de una escucha activa en lugar de un fallo.

ALCANCE DE LA VERIFICACIÓN

Sobre el emulador se comprobaron la solicitud del permiso de micrófono, el estado de escucha en pantalla, los mensajes de error legibles, el reintento automático descrito arriba y la liberación de los motores al salir. La conversión efectiva de voz a texto se verificó después sobre un teléfono físico, que es lo que un emulador sin micrófono no permite: dictada la pregunta «cuánto llevo ahorrado», la aplicación la transcribió al campo de consulta, mostró la respuesta y la leyó en voz alta. El caso queda registrado como 5-17 en la Sección 19, con la traza de los dos servicios del sistema.

Esa verificación dejó constancia de un comportamiento que conviene documentar. El sintetizador solicita la voz colombiana es-CO y, si no está instalada, la castellana es-ES; en el dispositivo de prueba no había ninguna de las dos y Android resolvió a es-US, la voz española más próxima entre las disponibles. La cadena de reserva no llegó a agotarse en un fallo porque el sistema operativo interpreta la petición como

```

{
  "agent_id": "nexcode-finance-agent",
  "periodo": "2026-08-01 a 2026-08-24",
  "resumen": {
    "ingresos": 1400000.0,
    "gastos": 294600.0,
    "saldo": 1105400.0,
    "tasa_ahorro": 0.78
  },
  "categorias": [
    { "nombre": "Mercado", "total": 87500.0, "movimientos": 1, "porcentaje": 29 },
    { "nombre": "Ropa", "total": 55000.0, "movimientos": 1, "porcentaje": 18 }
  ],
  "pregunta_usuario": null
}

```

Fragmento 16. *Payload generado por NexcodeAIService.buildPayload() con los datos de la verificación. Las claves van en español porque el agente responde al usuario en ese idioma.*

El envío está implementado con `URLConnection`, incluye cabecera de autorización opcional y tiempo de espera configurable, y su comportamiento ante fallo es degradar al

```

// app/di/AppContainer.kt – estado actual
val financialAdvisor: FinancialAdvisor = NexcodeAIService(
    NexcodeAIService.AiConfig(webhookUrl = null) // motor local
)

```

ESTADO REAL DE LA INTEGRACIÓN

El agente conversacional remoto **no está conectado**. Lo implementado y verificado es el asistente de voz sobre las API del sistema, el motor de reglas local, el contrato del dominio, la serialización del payload y el cliente HTTP con su degradación ante fallo. Lo que falta es crear el agente en la plataforma y proporcionar su dirección; no requiere cambios de código en la aplicación.

Se documenta así porque afirmar una integración que no opera sería falso, y porque la distinción entre «preparado» e «integrado» es precisamente lo que un revisor debe poder comprobar.

una preferencia y no como un requisito. El resultado es correcto en idioma, con un acento que no es el local: una degradación silenciosa preferible a un error.

La síntesis se resolvió además con una voz **incorporada al dispositivo**, no descargada en el momento. La aplicación conserva así su premisa en la modalidad hablada: el análisis se calcula en el dominio local, se pronuncia con un motor local y ningún dato financiero sale del teléfono.

E. Preparación para un agente conversacional

La integración con un agente externo —del tipo Retell [21] AI— está construida hasta el punto exacto donde empieza a depender de un servicio de terceros. El método `buildPayload` serializa la petición del dominio al formato JSON que viajaría al webhook, y es público a propósito: permite inspeccionar y documentar el contrato sin necesidad de tener el servicio contratado.

motor local anteponiendo la advertencia «(Sin conexión con el agente)». Activarlo requiere una sola línea: proporcionar la dirección del webhook en la configuración del servicio.

F. Asistencia de IA en el desarrollo

Corresponde declarar también el otro sentido en que la inteligencia artificial participó en el proyecto: como herramienta de desarrollo. Se emplearon asistentes de programación para la generación y revisión de código, la exploración de alternativas de diseño y la redacción de este informe. Las decisiones arquitectónicas, la selección de versiones y la verificación sobre dispositivo fueron ejecutadas y comprobadas por los autores, y cada afirmación técnica de

este documento está respaldada por una salida de consola reproducible.

XIX. PRUEBAS DE FUNCIONAMIENTO

La arquitectura descrita en la Sección 11 condiciona la estrategia de pruebas y la simplifica. Como la capa de dominio no depende de Android, sus reglas de negocio se verifican en la máquina virtual de Java sin arrancar un emulador, y el ciclo de retroalimentación se mide en segundos. Lo que sí requiere dispositivo son los comportamientos que dependen del sistema operativo: la persistencia real, los gestos y las notificaciones. De ahí los tres niveles de la Tabla XXXI, cada uno con su alcance y su costo de ejecución.

Tabla XXXI NIVELES DE VERIFICACIÓN APLICADOS				
Nivel	Qué verifica	Herramienta	Requiere dispositivo	Duración
Unitario	Reglas de negocio y cálculos del dominio	kotlin.test	No	Segundos
Integración	Persistencia real y actualización reactiva	App sobre Room	Sí	Minutos
Sistema	Flujos completos, gestos y notificaciones	adb · uiautomator	Sí	Minutos

A. Nivel unitario

Las 24 pruebas viven en el conjunto `commonTest` del módulo de dominio, lo que significa que se compilan y ejecutan dos veces: una para JVM y otra para Node.js. Cada ciclo de verificación produce por tanto 48 ejecuciones. La Tabla XXXII detalla los casos.

Tabla XXXII CASOS DE PRUEBA UNITARIOS				
ID	Caso	Qué comprueba	JVM	JS
U-01	Formato colombiano con miles	"12.500,50" → 12 501 pesos	OK	OK
U-02	Punto como separador decimal	"12500.50" → 12 501 pesos	OK	OK
U-03	Punto con tres dígitos es miles	"25.000" → 25 000 pesos	OK	OK
U-04	Símbolo de peso y espacios	" \$ 5000 " se normaliza	OK	OK
U-05	Monto nulo	Lanza <code>InvalidAmountException</code>	OK	OK
U-06	Texto no numérico	"mil pesos" es rechazado	OK	OK
U-07	Redondeo hacia arriba	"12,60" → 13 pesos	OK	OK
U-08	Redondeo hacia abajo	"12,40" → 12 pesos	OK	OK
U-09	Límite superior	Rechaza montos de diez cifras	OK	OK
U-10	Suma sin pérdida	10.000 + 20.000 + 5.500 da exactamente 35.500	OK	OK
U-11	Formato sin decimales	125 000 000 → "1.250.000"	OK	OK
U-12	Formato de un dato heredado	125 000 050 → "1.250.001"	OK	OK
U-13	Registro de gasto válido	Se guarda con el monto correcto	OK	OK
U-14	Uso del reloj inyectado	La fecha guardada es la del reloj fijo	OK	OK
U-15	Monto no numérico en el caso de uso	Devuelve <code>InvalidAmount</code>	OK	OK
U-16	Monto negativo	Devuelve <code>InvalidAmount</code>	OK	OK
U-17	Cuenta inexistente	Devuelve <code>NotFound</code>	OK	OK
U-18	Título vacío	Toma el nombre de la categoría	OK	OK
U-19	Nota en blanco	Se guarda como nulo, no como cadena vacía	OK	OK
U-20	Agrupación por categoría	Suma correcta y conteo de movimientos	OK	OK
U-21	Orden por gasto	De mayor a menor	OK	OK
U-22	Porcentajes	Suman 1 con tolerancia de 0,0001	OK	OK
U-23	Movimientos sin categoría	Se agrupan bajo «Sin categoría»	OK	OK
U-24	Exclusión de ingresos	No entran en el reparto de gastos	OK	OK

```
$ ./gradlew :domain:jvmTest :domain:jsNodeTest
```

```
jvmTest    · MoneyTest ..... 12 pruebas, 0 fallos
jvmTest    · AddTransactionUseCaseTest ..... 7 pruebas, 0 fallos
jvmTest    · GetCategorySummaryUseCaseTest .. 5 pruebas, 0 fallos
jsNodeTest · MoneyTest ..... 12 pruebas, 0 fallos
jsNodeTest · AddTransactionUseCaseTest ..... 7 pruebas, 0 fallos
jsNodeTest · GetCategorySummaryUseCaseTest .. 5 pruebas, 0 fallos
```

BUILD SUCCESSFUL

Salida 2. Ejecución de la batería unitaria en las dos plataformas.

Conviene señalar una decisión de diseño que hizo posible el caso **U-14**. Mientras el caso de uso llamaba directamente a `LocalDateTime.now()` era imposible verificar la fecha guardada, porque el valor esperado cambiaba en cada ejecución. Al inyectar el reloj como una dependencia (Sección 17), la prueba pasó a ser determinista. La testabilidad no fue un añadido posterior: fue consecuencia de una decisión arquitectónica.

B. Defecto detectado en el nivel unitario

El caso **U-03** merece registro aparte porque no se escribió para confirmar un comportamiento sino porque lo descubrió. Al redactar la prueba del caso de uso de registro con el valor "25.000", la aserción falló: el sistema guardaba **25 pesos** en lugar de veinticinco mil.

La causa era una regla de análisis ambigua que interpretaba todo punto único como separador decimal. El defecto no producía ningún error visible —la aplicación funcionaba y guardaba un valor incorrecto—, de modo que solo una aserción sobre el valor esperado podía detectarlo. La corrección y la prueba de regresión quedan documentadas en la Sección 16.

C. Cambio posterior: la aplicación opera en pesos enteros

Concluida la primera ronda de verificación se adoptó una decisión que afecta a todos los importes: la aplicación **dejó de manejar centavos**. El motivo es de contexto y no técnico —en Colombia el centavo no circula, y ningún usuario escribe «\$ 25.000,00»—, pero la forma de aplicarlo sí fue una decisión técnica con consecuencias verificables.

La alternativa sencilla habría sido ocultar los decimales únicamente en la presentación. Se descartó porque introduce una discrepancia silenciosa: el usuario escribiría 25.000,50, la pantalla mostraría 25.001 y la base guardaría un tercer valor. En su lugar el redondeo se trasladó al análisis del texto, dentro del propio dominio, de modo que **lo que el usuario escribe es**

exactamente lo que queda almacenado. Los casos **U-07** y **U-08** fijan ese comportamiento en las dos direcciones del redondeo, y **U-12** comprueba que un dato guardado antes del cambio se sigue mostrando de forma coherente.

El almacenamiento interno se mantuvo en centavos deliberadamente. Cambiarlo habría exigido una tercera migración de esquema sobre datos existentes —con riesgo real de pérdida— a cambio de ningún beneficio observable, y habría cerrado la puerta a una moneda que sí use fracciones. La representación interna y la unidad de trabajo del usuario son dos decisiones separadas, y conviene que sigan siéndolo.

OBSERVACIÓN

Un defecto que no lanza excepción y no rompe la interfaz es invisible para la prueba manual. Los doce casos de Money existen precisamente porque el análisis de texto a número es el punto donde un error se convierte en un dato incorrecto y silencioso en la base de datos.

D. Nivel de sistema

Los casos de sistema se ejecutaron sobre **dos dispositivos distintos**, elegidos para cubrir los extremos del rango que la aplicación declara soportar: un emulador Pixel 6 con Android API 35 y un teléfono Motorola Moto G Power con Android 11 (API 30). La duplicidad no es redundante: entre esas dos versiones cambia el comportamiento del sistema operativo en un punto que el código debe contemplar, como se detalla tras la tabla.

La evidencia no se recogió por inspección visual sino por instrumentación: `uiautomator dump` entrega el árbol de la interfaz con el texto real en pantalla, `dumpsys notification` el contenido efectivo de las notificaciones publicadas y `logcat` la traza de los servicios del sistema. La Tabla XXXIII recoge los diecisiete casos.

Tabla XXXIII
CASOS DE PRUEBA DE SISTEMA SOBRE DISPOSITIVO

ID	Caso	Resultado esperado	Obtenido	Estado
S-01	Primer arranque	Siembra de 10 cuentas, 10 categorías, 10 movimientos y 10 pagos fijos	Datos visibles en las cuatro secciones	OK
S-02	Registrar un gasto	"25.000" se guarda como \$ 25.000	Saldo actualizado y fila creada bajo «Hoy»	OK
S-03	Guardar con monto vacío	Mensaje de error, sin registro creado	Texto "El monto no puede estar vacío" en el árbol de la interfaz	OK
S-04	Título vacío	Se usa el nombre de la categoría	Movimiento titulado «Comida»	OK
S-05	Editar un movimiento	\$ 12.000 → \$ 45.500 con recálculo en cadena	Saldo 1.138.900 → 1.105.400; gastos 261.100 → 294.600; ahorro 81% → 78%	OK
S-06	Recálculo del gráfico	La dona refleja el movimiento editado	«Comida» entra al reparto con 15%	OK

S-07	Eliminar deslizando	El movimiento desaparece de la lista	Fila retirada; lista reordenada con animación	OK
S-08	Pagar un recordatorio	Se crea el gasto y avanza la fecha	Movimiento «Energía» de \$ 120.000 creado; contador «hoy» 1 → 0; insignia 2 → 1	OK
S-09	Permiso de notificaciones	Se solicita en el primer arranque en Android 13 o superior	Diálogo mostrado en API 35; en API 30 no se solicita y la notificación sale igual	OK
S-10	Notificación de vencimientos	Aviso con los pagos vencidos y de hoy	«3 pagos fijos por atender · Suman \$ 1.005.000» con detalle por pago	OK
S-11	Navegación desde la notificación	Abre el Bolsillo, no el Inicio	Pantalla «Bolsillo de recordatorios» en primer plano	OK
S-12	Asistente financiero por texto	Analiza el mes y entrega consejos	«Comida concentra el 100% de tus gastos» y dos consejos más	OK
S-13	Permiso de micrófono	Se solicita al tocar el botón de voz por primera vez	Diálogo «Allow Nexcode to record audio?» mostrado	OK
S-14	Estado de escucha	La pantalla indica que el micrófono está abierto	Tarjeta «Escuchando... / Habla ahora» con el micrófono latiendo	OK
S-15	Reintento sin paquete de idioma	Un fallo del motor local no aborta la consulta	Error 12 registrado y escucha activa en el segundo intento	OK
S-16	Liberación de los motores	Al salir de la pantalla se cierran reconocedor y sintetizador	Registro: «Disconnected from TTS engine» y «RecognitionService#onDestroy»	OK
S-17	Consulta hablada de extremo a extremo	Se dicta una pregunta y el asistente responde en pantalla y en voz alta	«cuánto llevo ahorrado» transcrito al campo; respuesta mostrada y leída con voz local	OK

LO QUE APORTÓ EL SEGUNDO DISPOSITIVO

El caso S-09 se comporta de forma distinta en cada versión y por eso justifica la doble ejecución. El permiso POST_NOTIFICATIONS no existe antes de Android 13; en el emulador con API 35 el sistema muestra el diálogo de autorización, mientras que en el teléfono con API 30 la solicitud se omite y la notificación se publica igualmente. Son dos caminos de código distintos gobernados por una comprobación de versión, y probar solo uno habría dejado el otro

sin verificar. Es el argumento práctico a favor de no confiar la verificación a un único dispositivo.

El caso **S-03** es el que cierra el ciclo del manejo de excepciones descrito en la Sección 14. La evidencia recogida no es una captura de pantalla sino el nodo de texto real del árbol de la interfaz, lo que demuestra que el mensaje nace en el dominio y llega intacto a la vista sin que ninguna excepción se propague:

```
$ adb shell uiautomator dump /sdcard/ui.xml && adb shell cat /sdcard/ui.xml
```

```
<node index="10" text="El monto no puede estar vacio"
  class="android.widget.TextView"
  package="com.nexcode.gastos"
  bounds="[53,1655][545,1718]" />
```

Salida 3. Caso S-03 — el mensaje del dominio, capturado del árbol de vistas del dispositivo.

```
$ adb shell dumpsys notification --noredact
```

```
NotificationRecord(pkg=com.nexcode.gastos id=1001 importance=4
  channel=recordatorios_pagos_fijos category=reminder)
  android.title = "3 pagos fijos por atender"
  android.text = "Suman $ 1.005.000"
    Arriendo · $ 850.000 · vencido hace 3 días
    Energía · $ 120.000 · vence hoy
    Pasajes de la semana · $ 35.000 · vence hoy
```

Salida 4. Caso S-10 — contenido efectivo de la notificación publicada por el trabajo en segundo plano.

El caso **S-15** comprueba una recuperación y no un camino feliz: el dispositivo de prueba carece del paquete de español para reconocimiento sin conexión, de modo que el primer

intento falla siempre. La evidencia de que el reintento funciona es la secuencia del registro seguida del estado de la pantalla:

```
$ adb logcat | grep ReconocedorDeVoz
W ReconocedorDeVoz: Error de reconocimiento: codigo 12

$ adb shell uiautomator dump && adb shell cat /sdcard/window_dump.xml
text="Escuchando..." text="Habla ahora"
```

Salida 1. Caso S-15 — el motor local falla por falta de paquete de idioma y el reintento en línea deja el micrófono escuchando.

El caso **S-17** es el único que no podía ejecutarse sobre el emulador, porque requiere una entrada de micrófono real. Se ejecutó sobre el teléfono físico dictando la pregunta «cuánto

llevo ahorrado». La evidencia no es la captura de pantalla — que solo probaría que apareció un texto— sino la traza de los

dos servicios del sistema implicados, que demuestra que el reconocimiento y la síntesis se ejecutaron de verdad:

```
$ adb logcat | grep -E "RecognitionService|GoogleTTSService"
```

```
RecognitionService#onStartOfSpeech
RecognitionService#onEndOfSpeech
RecognitionService#onEndRecognitionSession
GoogleTTSServiceImpl: Synthesis request for locale spa-USA and name es-US-language
GoogleTTSServiceImpl: TTS dispatch: es-us-x-esc-seanet-embedded
```

Salida 5. Caso S-17 sobre el teléfono físico — el reconocedor abre y cierra la sesión de escucha, y el sintetizador atiende la petición de lectura. El sufijo *embedded* indica que la voz se generó en el propio dispositivo, sin conexión.

La respuesta obtenida fue: «Sobre “cuánto llevo ahorrado”: Este mes ingresaste 1.400.000 y gastaste 261.100. Tu saldo es +1.138.900». El asistente cita la pregunta transcrita, lo que permite al usuario detectar de inmediato un error de reconocimiento sin necesidad de volver a dictarla.

La traza deja también constancia de una **degradación aceptada**: el sintetizador resolvió a es-US y no a es-CO ni es-ES, porque el dispositivo no tenía instalada ninguna de esas dos voces. El código solicita la variante colombiana y Android entrega la voz castellana más próxima disponible en lugar de fallar. El resultado es correcto en idioma y comprensible, con un acento que no es el local.

E. Verificación de ausencia de fallos

Además de comprobar lo que debe ocurrir, tras cada instalación se verificó que no ocurriera lo que no debe. Se inspeccionó el registro del sistema en busca de excepciones fatales y se ejecutó el análisis estático sobre el proyecto completo. La Tabla XXXIV resume estas comprobaciones transversales.

Tabla XXXIV
COMPROBACIONES TRANSVERSALES

Comprobación	Instrumento	Resultado
Ausencia de excepciones fatales	adb logcat	0 coincidencias de FATAL EXCEPTION
Análisis estático	./gradlew lint	Sin errores
Construcción completa	./gradlew build	BUILD SUCCESSFUL
Aislamiento de capas	Compilador de Kotlin	Violación deliberada rechazada
Portabilidad del dominio	:domain:jsNodeTest	24/24 en Node.js

FALSO POSITIVO REGISTRADO

Durante el caso S-07 la aplicación desapareció de la pantalla y quedó visible otra aplicación, lo que sugirió un fallo grave en el componente de deslizamiento. La inspección de la evidencia mostró que el gesto de prueba, iniciado en el borde derecho de la pantalla, había activado el gesto «atrás» del sistema operativo. Repetido lejos del borde, el borrado funcionó correctamente. **El defecto estaba en el método de prueba, no en el código**; se documenta porque descartar hipótesis forma parte del proceso de verificación.

F. Cobertura y limitaciones

La cobertura no es uniforme por diseño, y conviene declarar dónde es fuerte y dónde no. La capa de dominio —donde vive la lógica que puede producir datos incorrectos— está cubierta por pruebas automatizadas que se ejecutan en cada construcción. Las capas de datos y presentación se verificaron mediante casos de sistema manuales sobre dispositivo.

La única limitación que quedaba pendiente en una versión anterior de esta sección —la imposibilidad de comprobar la conversión efectiva de voz a texto sobre un emulador sin micrófono— **quedó resuelta con la ejecución del caso S-17 sobre el teléfono físico**. Se deja constancia del recorrido porque ilustra un principio: una limitación declarada es una tarea pendiente identificada, no una excusa, y el instrumento adecuado para levantarla no fue más código sino el dispositivo correcto.

Las limitaciones reconocidas son tres. **No hay pruebas instrumentadas [33] automatizadas**: los diecisiete casos de la Tabla XXXIII se ejecutaron de forma manual asistida por adb, de modo que no forman parte del ciclo de integración continua y deben repetirse a mano. **No hay pruebas de la capa de datos con base en memoria**, que verificarían los DAO y la migración de esquema de forma automatizada. Y **no se midió la cobertura de código**; las cifras presentadas son de casos ejecutados, no de porcentaje de líneas alcanzadas.

El trabajo futuro más rentable en este frente sería incorporar Espresso o Compose UI Test para convertir los casos S-02 a S-08 en pruebas automatizadas, y una base Room en memoria para probar la migración 1→2 sin dispositivo.

XX. RESULTADOS

El resultado del proyecto es una aplicación Android funcional, instalada y verificada sobre dispositivo, cuya capa de lógica de negocio compila además para una segunda plataforma. Esta sección presenta primero las magnitudes del producto, después el cumplimiento de los objetivos planteados en la Sección 5, y por último los hallazgos que no se esperaban al iniciar.

A. Magnitudes del producto

MAGNITUDES DEL PRODUCTO

7 631 líneas de Kotlin	80 archivos fuente	3 módulos independientes
17 casos de uso	24 pruebas unitarias	2 plataformas de ejecución
17	6	0

casos de sistema	pantallas	servicios externos de pago
2 versiones de Android verificadas	1,7 MB APK de publicación	

Tabla XXXV
COMPOSICIÓN DEL PRODUCTO ENTREGADO

Elemento	Cantidad	Detalle
Módulos Gradle	3	:app :domain :data
Archivos por capa	29 / 18 / 33	Dominio / Datos / Presentación
Modelos de dominio	8	Incluye dos jerarquías selladas
Casos de uso	17	Transacciones, análisis, bolsillo, catálogo e IA
Contratos de repositorio	5	Cuatro de datos y uno de asistencia
Tablas de la base	4	Esquema en versión 2 con migración escrita
Pantallas	6	Cuatro destinos y dos pantallas completas
Componentes propios	8	Reutilizados en varias pantallas
Animaciones	12	Cada una con una

		función comunicativa
Modalidades del asistente	2	Voz y texto sobre el mismo caso de uso
Datos de demostración	40	10 por cada una de las cuatro secciones
Tamaño del APK firmado	1,7 MB	Con R8 y recorte de recursos; 11,9 MB sin optimizar
Reglas de conservación	4	Bloques en proguard-rules.pro, cada uno justificado

B. Cumplimiento de los objetivos

La Tabla XXXVI contrasta cada objetivo específico de la Sección 5 con la evidencia obtenida. Los seis se cumplieron. Ello no significa que el producto esté acabado: el apartado C de la Sección 21 recoge siete limitaciones de alcance, y ninguna de ellas contradice un objetivo planteado.

Tabla XXXVI
CUMPLIMIENTO DE LOS OBJETIVOS ESPECÍFICOS

N.º	Objetivo	Evidencia obtenida	Estado
O-1	Arquitectura en tres capas independientes	La violación deliberada produjo BUILD FAILED; cero imports de Android en el dominio; un único acceso de la presentación a la capa de datos	Cumplido
O-2	Registro, edición y consulta con persistencia local	Casos S-02 a S-07 superados; recálculo en cadena verificado sobre dispositivo	Cumplido
O-3	Bolsillo con aviso del sistema operativo	Notificación publicada con la app cerrada; navegación desde ella al Bolsillo; retirada automática al tocarla	Cumplido
O-4	Asistente que analice el gasto	Asistente operativo por voz y por texto sobre datos reales (casos S-12 y S-13). El agente conversacional remoto queda preparado y no conectado, lo que se declara como limitación de alcance y no como incumplimiento: el objetivo no lo exigía	Cumplido
O-5	Evidenciar los pilares de programación	15 fragmentos citados literalmente con su archivo de origen en las secciones 12 a 16	Cumplido
O-6	Verificar con pruebas automatizadas	24 pruebas en JVM y las mismas 24 en Node.js, sin fallos; 17 casos de sistema con evidencia instrumentada sobre dos versiones de Android	Cumplido

C. Resultados no previstos

Cuatro resultados no figuraban entre los objetivos y merecen registro porque tuvieron consecuencias sobre el producto.

La portabilidad del dominio. La migración a Kotlin Multiplatform no estaba planteada al inicio; se emprendió para poder responder con evidencia a la pregunta de si la capa de dominio era realmente independiente. El resultado fue afirmativo, pero solo después de corregir dos acoplamientos a la biblioteca de Java que la revisión manual no había

detectado. La verificación produjo, además de la respuesta, una mejora del código.

Un defecto de lógica silencioso. El análisis del texto "25.000" almacenaba veinticinco pesos en lugar de veinticinco mil. No producía error alguno: la aplicación funcionaba y guardaba un dato incorrecto. Lo detectó una prueba unitaria antes de que ningún usuario lo encontrara, lo que justifica por sí solo la inversión en la batería de pruebas del dominio.

Lo que el optimizador no puede ver. Al preparar la compilación de publicación se activó R8 [34], que elimina el código no alcanzado y renombra el resto: el paquete instalable pasó de 11,9 MB a **1,7 MB**, algo menos de la séptima parte. La

reducción no fue el hallazgo. Lo fue advertir que el análisis de R8 sigue llamadas explícitas y, por tanto, **es ciego a todo lo que se instancia por reflexión**. En esta aplicación había un caso: WorkManager guarda el nombre de la clase del trabajador en su propia base de datos y la construye al despertar, de modo que R8 no ve ninguna llamada a su constructor y lo habría renombrado. El fallo resultante no aparece al compilar —la construcción termina con éxito— sino en el dispositivo, y de forma silenciosa: la notificación sencillamente deja de publicarse. La regla de conservación escrita para ese constructor se verificó comprobando que el registro del sistema imprime el nombre completo sin ofuscar, seguido de `Worker result SUCCESS`. El caso ilustra una asimetría útil: una herramienta de análisis estático solo protege aquello cuyo uso puede demostrar, y el resto queda a cargo de quien conoce el código.

El compilador como revisor de arquitectura. Durante la integración, el error `Cannot access 'androidx.room.RoomDatabase'` señaló que la capa de presentación estaba construyendo la base de datos. La solución correcta no era añadir la dependencia que faltaba —lo que habría hecho desaparecer el error y también la separación de capas— sino mover el ensamblaje a la capa que lo posee. La arquitectura quedó más estricta después del error que antes.

XXI. CONCLUSIONES

A. Sobre la pregunta problema

La pregunta planteada en la Sección 6 tenía tres componentes y admite hoy una respuesta diferenciada.

Operar sin cuenta ni conexión quedó resuelto sin reservas: la totalidad de las funciones opera contra una base de datos local, no existe registro de usuario y ningún dato abandona el dispositivo. La contrapartida —ausencia de respaldo y de sincronización— es real y se documenta como limitación.

El asistente financiero, por su parte, terminó admitiendo **voz además de texto** sin tocar la capa de dominio: el reconocimiento y la síntesis del sistema operativo se conectaron al mismo caso de uso que ya atendía la entrada escrita. Es la prueba más práctica de que la separación de capas rinde: una modalidad de interacción nueva no obligó a modificar una sola regla de negocio.

Recordar activamente los pagos fijos quedó resuelto en el sentido pleno del verbo: la aplicación avisa aunque esté cerrada, mediante un trabajo programado que consulta el dominio y publica una notificación del sistema. Antes de esa incorporación, el bolsillo mostraba estado pero no recordaba nada, y un recordatorio que exige ser consultado no es un recordatorio.

Que la arquitectura sea verificable y no solo afirmada es el componente que produjo el aprendizaje más sólido. Se dispuso de cinco verificaciones reproducibles, y la más concluyente resultó ser también la más incómoda: compilar la capa de dominio para JavaScript reveló dos dependencias de Java que ninguna revisión manual había advertido. La conclusión que se extrae excede el proyecto: **una propiedad**

arquitectónica que no se comprueba de forma automática tiende a degradarse sin que nadie lo note.

B. Sobre las decisiones técnicas

Cinco decisiones se confirmaron acertadas al ponerlas a prueba.

Un módulo por capa en lugar de paquetes dentro de un módulo único. El costo —más configuración de construcción, una operación que atraviesa más capas— se pagó una sola vez, mientras que el beneficio se cobró en cada verificación posterior. Con paquetes, la violación de la Sección 11 habría compilado sin protestar.

Resultados sellados en lugar de excepciones hacia la capa de presentación. La interfaz nunca envuelve una llamada en un bloque `try`: recibe un estado exhaustivo que el compilador la obliga a cubrir por completo.

Importes en enteros dentro de un tipo propio. Evitó por diseño los errores de coma flotante, y el tipo específico impidió operaciones que no tienen sentido sobre dinero. La decisión mostró además su valor cuando la aplicación pasó a operar en pesos enteros: como todo importe atraviesa un único tipo, el cambio se aplicó en dos funciones del dominio y se comprobó con dos pruebas, sin tocar una sola pantalla ni migrar la base de datos.

Injectar el reloj en lugar de consultarlo directamente. Nació como requisito de portabilidad y terminó habilitando una prueba que antes era imposible: verificar la fecha con la que se guarda un movimiento.

Verificar sobre dos versiones de Android y no solo sobre el emulador de desarrollo. El teléfono con API 30 confirmó el único caso que el emulador no podía ejecutar —la transcripción de voz real— y ejercitó un camino de código distinto en las notificaciones, donde el permiso que Android 13 exige sencillamente no existe. Probar en un solo dispositivo habría dejado ambos sin comprobar.

Una decisión, en cambio, resultó equivocada y hubo que rehacerla. La primera versión de la navegación desde la notificación leía el destino solamente en `onCreate`, lo que funcionaba con la aplicación cerrada y fallaba con la aplicación abierta, porque en ese caso Android entrega el nuevo intento a `onNewIntent`. El error enseña que el ciclo de vida de Android obliga a considerar los dos caminos de entrada a una pantalla, no solo el más frecuente.

C. Limitaciones reconocidas

Se declaran de forma explícita, ordenadas por su impacto sobre un usuario real.

Tabla XXXVII
LIMITACIONES DEL ENTREGABLE

Limitación	Consecuencia para el usuario	Causa
Sin respaldo ni exportación	Desinstalar la aplicación borra el historial	Recorte de alcance
Sin presupuesto por categoría	No hay alerta al aproximarse a un límite	Recorte de alcance; el modelo de dominio ya lo soporta
Agente conversacional sin conectar	El análisis es local y no conversacional	Requiere contratar el servicio externo
Sin objetivo iOS	No hay versión para	Compilar para iOS

	iPhone	exige un equipo con macOS
Interfaz no compartida	Una eventual versión web exigiría rehacer la interfaz	Compose Multiplatform quedó fuera de alcance
Pruebas de sistema manuales	Los 17 casos deben repetirse a mano en cada entrega	No se incorporó Espresso ni Compose UI Test
Solo tema claro	No hay modo oscuro	Decisión de producto; el sistema de color lo permitiría

D. Trabajo futuro

Las líneas de continuación se ordenan por la relación entre esfuerzo e impacto, evaluada sobre el estado actual del código.

- Presupuesto por categoría con alerta. Es la de mejor relación esfuerzo/impacto: el modelo Category ya declara el presupuesto mensual y la función que calcula su consumo está implementada y probada. Falta la pantalla y enlazarlo con el asistente, que podría avisar al superarse el límite.
- Exportación a CSV. Elimina la limitación de mayor impacto —la pérdida del historial— con un esfuerzo reducido, ya que la consulta por rango de fechas existe.
- Conectar el agente conversacional. No requiere cambios de código: crear el agente y proporcionar la dirección del webhook.
- Automatizar los casos de sistema con Compose UI Test, y añadir pruebas de la capa de datos con una base en memoria para verificar la migración de esquema sin dispositivo.
- Añadir el objetivo iOS al módulo de dominio. El código común no lo impide; la restricción es de infraestructura.

E. Conclusión general

El proyecto entrega una aplicación funcional que resuelve el problema planteado dentro del alcance definido, y lo hace con una arquitectura cuya corrección no depende de la palabra de sus autores. Esa es, en última instancia, la aportación que distingue este trabajo: no que la aplicación funcione —muchas funcionan— sino que cada afirmación técnica del informe va acompañada del comando que permite comprobarla.

Frente a las soluciones consolidadas del mercado, Nexcode no ofrece mayor cobertura funcional, y la Sección 8 lo documenta con una comparación desfavorable en varias características. Ofrece, en cambio, una combinación que ninguna de las alternativas revisadas reúne para este caso de uso: operar sin cuenta ni servidor, recordar activamente los pagos fijos y exponer una construcción auditable. Para el contexto académico en que se desarrolla, la última propiedad no es la menos importante: es el objeto mismo del ejercicio.

XXII. REFERENCIAS BIBLIOGRÁFICAS

- [1] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston, MA, EE. UU.: Prentice Hall, 2017.
- [2] Google, «Guide to app architecture», Android Developers. [En línea]. Disponible en: <https://developer.android.com/topic/architecture>
- [3] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, EE. UU.: Addison-Wesley, 2003.
- [4] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, EE. UU.: Addison-Wesley, 2002.
- [5] Google, «Save data in a local database using Room», Android Developers. [En línea]. Disponible en: <https://developer.android.com/training/data-storage/room>
- [6] JetBrains, «Kotlin coroutines and Flow documentation». [En línea]. Disponible en: <https://kotlinlang.org/docs/flow.html>
- [7] Aimlogic, «Monefy — Budget & Expense Manager». [En línea]. Disponible en: <https://monefy.me>
- [8] Realbyte Inc., «Money Manager Expense & Budget». [En línea]. Disponible en: <https://www.realbyteapps.com>
- [9] Rammig Software, «Bluecoins Finance & Budget». [En línea]. Disponible en: <https://www.bluecoinsapp.com>
- [10] Mobills, «Mobills — Control financiero». [En línea]. Disponible en: <https://www.mobills.com.co>
- [11] BudgetBakers, «Wallet by BudgetBakers». [En línea]. Disponible en: <https://budgetbakers.com>
- [12] Spendee, «Spendee — Money Tracker». [En línea]. Disponible en: <https://www.spendee.com>
- [13] You Need A Budget LLC, «YNAB — You Need A Budget». [En línea]. Disponible en: <https://www.ynab.com>
- [14] Actual Budget, «Actual — A local-first personal finance app». [En línea]. Disponible en: <https://actualbudget.org>
- [15] J. F. Grigg, «Firefly III — A free and open source personal finance manager». [En línea]. Disponible en: <https://www.firefly-iii.org>
- [16] Ivy Apps, «Ivy Wallet — Open source money manager». [En línea]. Disponible en: <https://github.com/Ivy-Apps/ivy-wallet>
- [17] Google, «Schedule tasks with WorkManager», Android Developers. [En línea]. Disponible en: <https://developer.android.com/topic/libraries/architecture/workmanager>
- [18] Google, «Jetpack Compose», Android Developers. [En línea]. Disponible en: <https://developer.android.com/jetpack/compose>
- [19] JetBrains, «Kotlin Multiplatform documentation». [En línea]. Disponible en: <https://kotlinlang.org/docs/multiplatform.html>
- [20] JetBrains, «kotlinx-datetime». [En línea]. Disponible en: <https://github.com/Kotlin/kotlinx-datetime>
- [21] Retell AI, «Retell AI — Voice agents platform». [En línea]. Disponible en: <https://www.retellai.com>
- [22] JetBrains, «Null safety», Kotlin documentation. [En línea]. Disponible en: <https://kotlinlang.org/docs/null-safety.html>
- [23] JetBrains, «Exceptions», Kotlin documentation. [En línea]. Disponible en: <https://kotlinlang.org/docs/exceptions.html>
- [24] JetBrains, «Sealed classes and interfaces», Kotlin documentation. [En línea]. Disponible en: <https://kotlinlang.org/docs/sealed-classes.html>
- [25] J. Gossman, «Introduction to Model/View/ViewModel pattern for building WPF apps», Microsoft Developer Blogs, oct. 2005. [En línea]. Disponible en: <https://learn.microsoft.com/archive/blogs/johngossman>
- [26] Gradle Inc., «Gradle User Manual». [En línea]. Disponible en: <https://docs.gradle.org/current/userguide/userguide.html>
- [27] JetBrains, «Inline value classes», Kotlin documentation. [En línea]. Disponible en: <https://kotlinlang.org/docs/inline-classes.html>
- [28] SQLite Consortium, «SQLite». [En línea]. Disponible en: <https://www.sqlite.org>
- [29] Google, «Migrate your Room database», Android Developers. [En línea]. Disponible en: <https://developer.android.com/training/data-storage/room/migrating-db-versions>
- [30] Google, «Notification runtime permission», Android Developers. [En línea]. Disponible en: <https://developer.android.com/develop/ui/views/notifications/notification-permission>
- [31] Google, «SpeechRecognizer», Android Developers API reference. [En línea]. Disponible en: <https://developer.android.com/reference/android/speech/SpeechRecognizer>
- [32] Google, «TextToSpeech», Android Developers API reference. [En línea]. Disponible en: <https://developer.android.com/reference/android/speech/tts/TextToSpeech>

- [33] Google, «Test apps on Android», Android Developers. [En línea]. Disponible en: <https://developer.android.com/training/testing>
- [34] Google, «Shrink, obfuscate, and optimize your app», Android Developers. [En línea]. Disponible en: <https://developer.android.com/build/shrink-code>